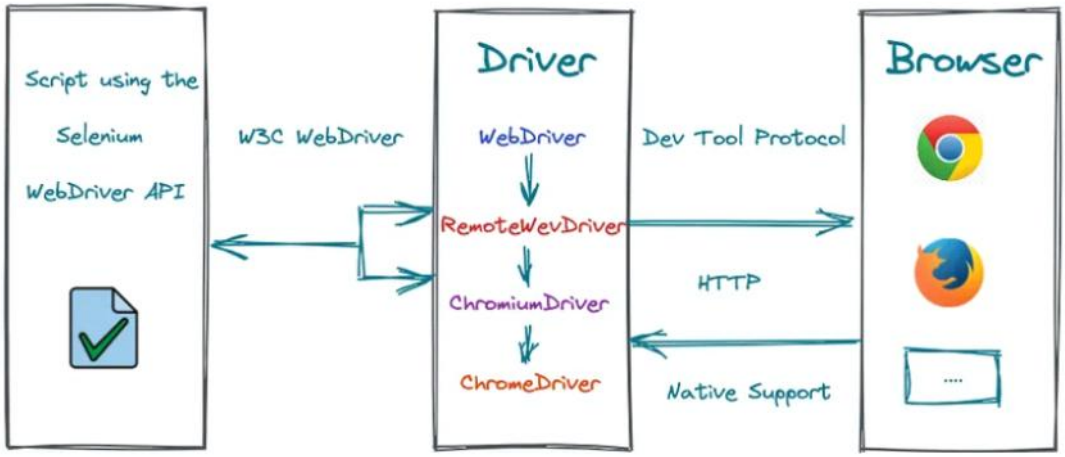
Introduction

Monday, May 21, 2018 12:32 PM

Selenium WebDriver is an open-source, cross-platform library designed to help automate browser testing. It defines set of methods but does not implements it.

How it Works

- You write your test script in your preferred language using methods like findElement provided by the language binding.
- The language binding translates these methods into commands that align with the WebDriver interface.
- The WebDriver implementation sends these commands to the appropriate browser driver.
- The browser driver interacts with the targeted browser to perform the actions (finding elements, clicking buttons, etc.).



W3C WebDriver Protocol

W3C Webdriver

In Selenium 3.0, tests communicated with the browser at the end node through the JSON wire protocol at the local end, necessitating API encoding and decoding. In Selenium 4, tests directly communicate through the W3C Protocol without encoding or decoding API requests.

There is a direct information exchange between the client and server, without the need for the JSON Wire Protocol. As Selenium WebDriver and web browsers use the same protocol, automated Selenium testing will execute tests more consistently between different browsers. Flakiness in web automation can be considerably reduced if a standard protocol is used.

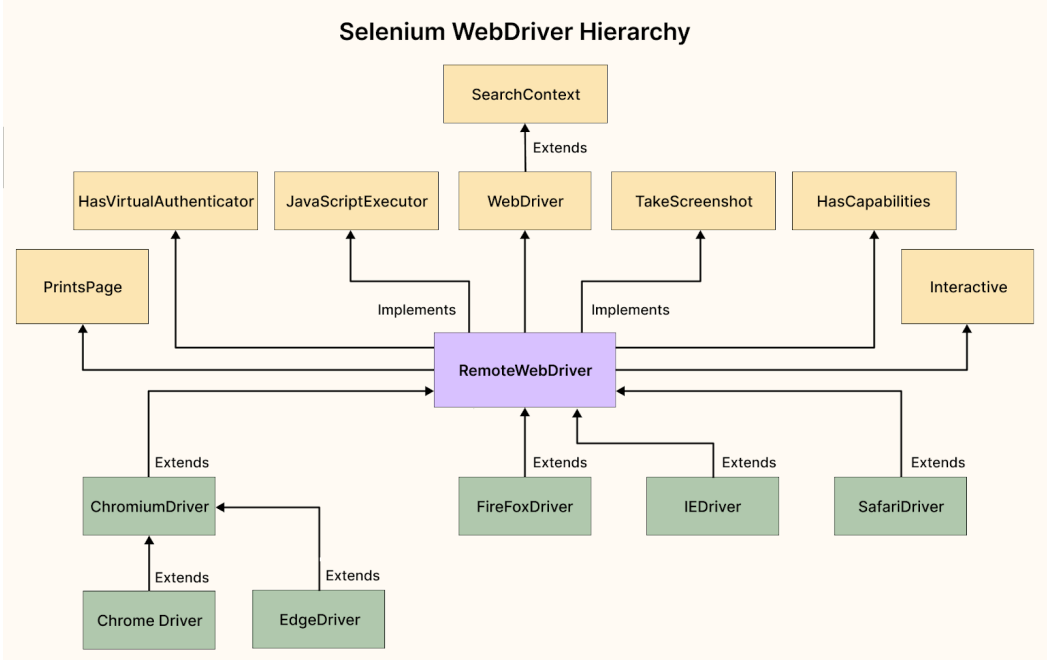
JSON wire protocol was replaced with W3C protocol because browers were following W3C standards, their executibles were also compliant to W3C standards

Browser Drivers

Selenium uses drivers, specific to each browser in order to establish a secure connection with the browser without revealing the internal logic of browser's functionality. The browser driver is also specific to the language used for automation such as Java, C#, etc.
When we execute a test script using WebDriver, the following operations are performed internally.

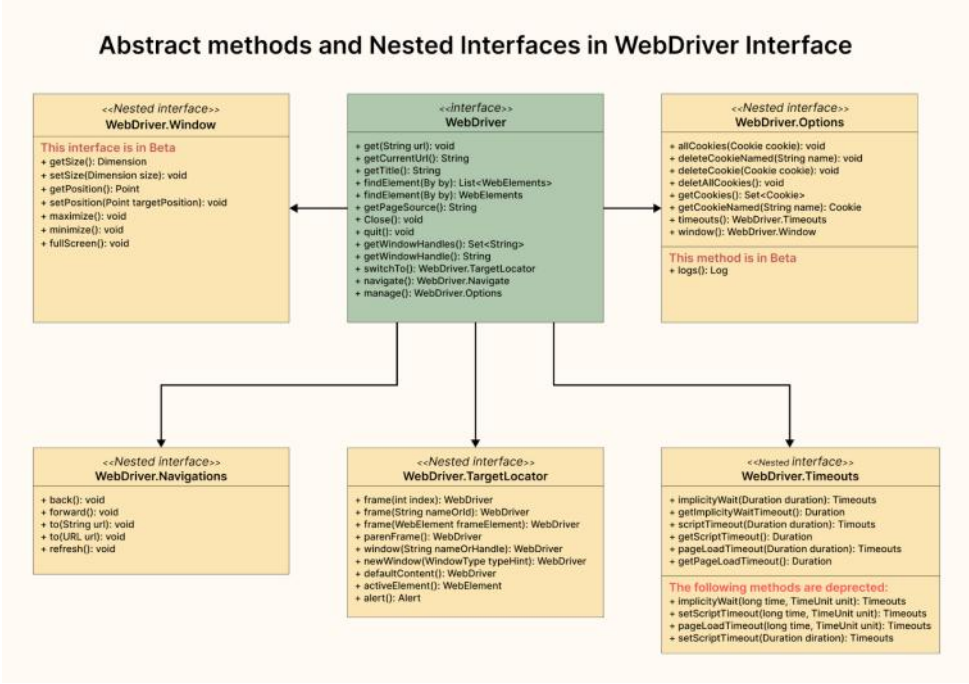
- HTTP request is generated and sent to the browser driver for each Selenium command.
- The driver receives the HTTP request through HTTP server.
- HTTP Server decides all the steps to perform instructions which are executed on browser.
- Execution status is sent back to HTTP Server which is subsequently sent back to automation script.

WebDriver Hierarchy



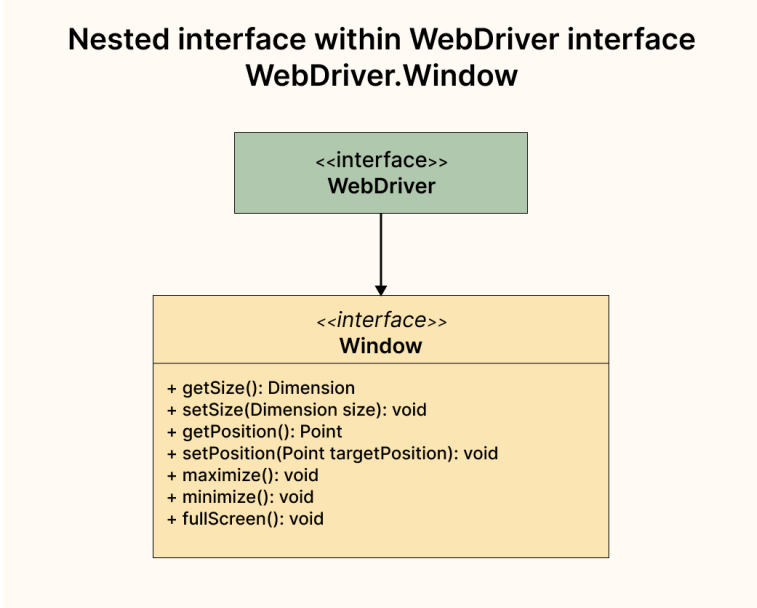
RemoteWebDriver Class	it is a fully implemented WebDriver Interface class extended by every BrowserDriver class within the Selenium framework. It enables you to execute your test scripts on remote machines or browsers Most Frequently used RemoteWebDriver Class Constructors: RemoteWebDriver(Uri, ICapabilities) RemoteWebdriver class implements the following interfaces:

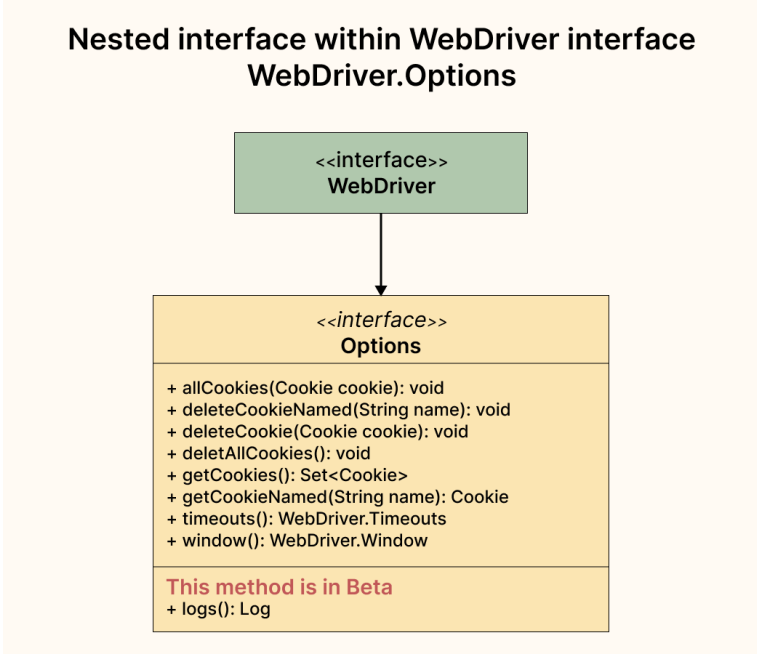
WebDriver Interface	WebDriver Interface is the core of the Selenium WebDriver as it has all the required methods and respective nested interfaces defined within it, which helps in simulating user actions inside the browser.

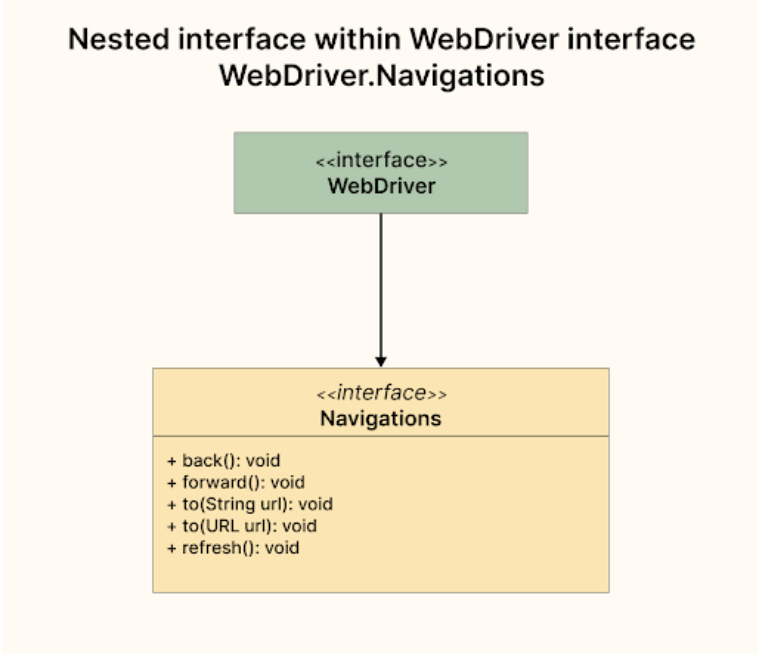


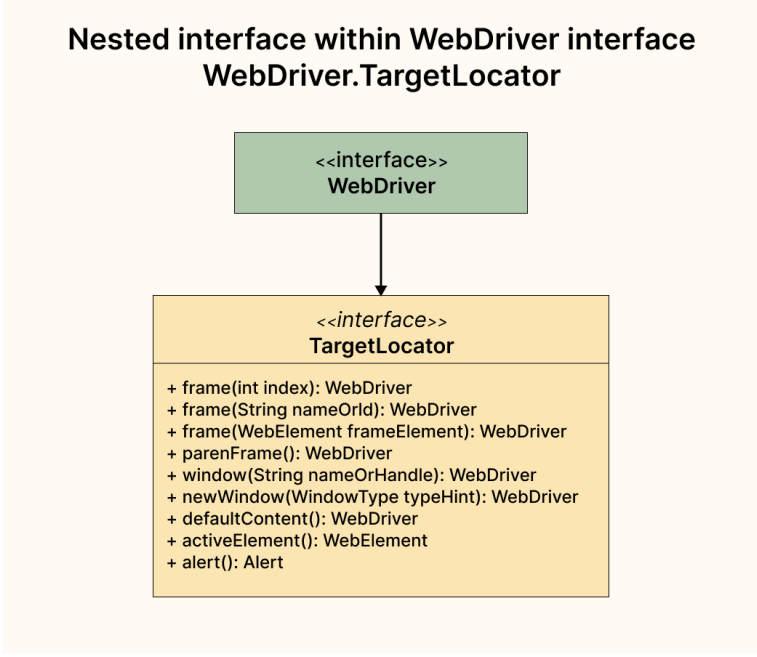
Nested Interfaces within WebDriver Interface

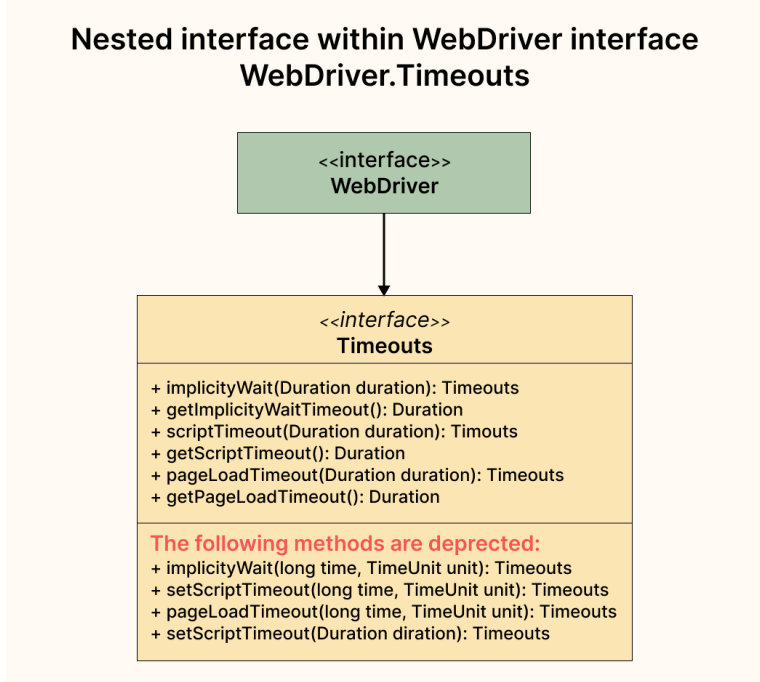
Window Interface	This interface has all the methods that help manage the current window. Usage: driver.manager().window().



Options Interface	This interface has all the methods to help manage the stuff in a browser menu. With the help of this interface, we perform the following actions: Add, get, and delete a cookie Set timeouts in the browser Manage window Fetch different types of logs. Usage: driver.manager().window(). 
Navigation Interface	This interface has all the methods to access the browser's history and navigate to a URL. With the help of this interface, we perform the following actions: Navigate Back, Forward in the browser Navigate to a URL in the browser Refresh the WebPage Usage: driver.navigate().



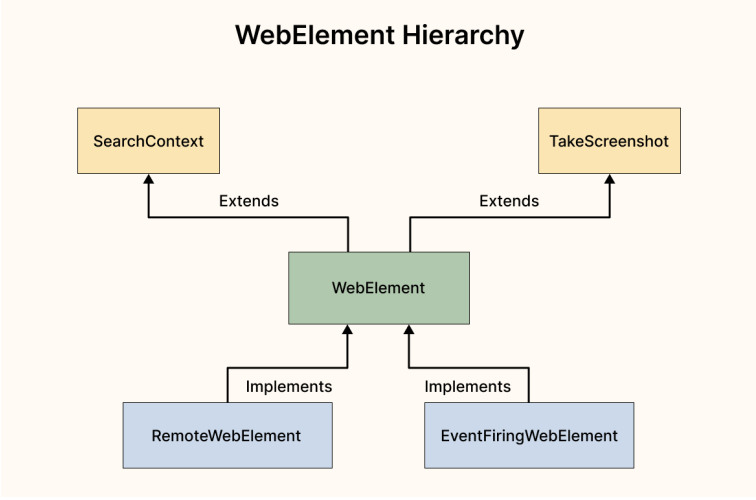
TargetLocator Interface	This interface has all the methods to send future commands to different frames and windows. With the help of this interface, we perform the following actions: Working with different Frames. Working with different windows or Tabs in the browser. Working with different Alerts in the browser. Usage: driver.switchTo(). 
Timeouts Interface	This interface has all the methods to manage the timeout behavior for WebDriver instances. With the help of this interface, we perform the following wait actions in Selenium: Implicit Wait Script timeout Page load timeout

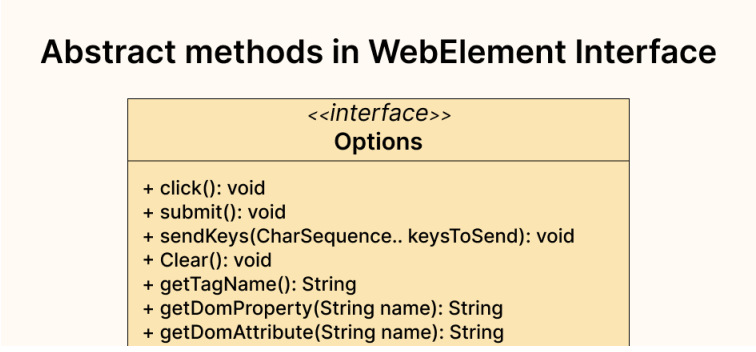


JavaScriptExecutor Interface	JavaScriptExecutor Interface provides the mechanism to WebDriver so that it can execute JavaScript code snippets
TakesScreenshot Interface	This interface helps the WebDriver take screenshots of the web page or WebElement as required and store them in different ways. The screenshot captured is returned to the WebDriver endpoint in Base64 format
HasVirtualAuthenticator Interface	This interface helps in allowing the WebDriver to access the virtual authenticator API.
PrintsPage Interface	
HasCapabilities Interface	

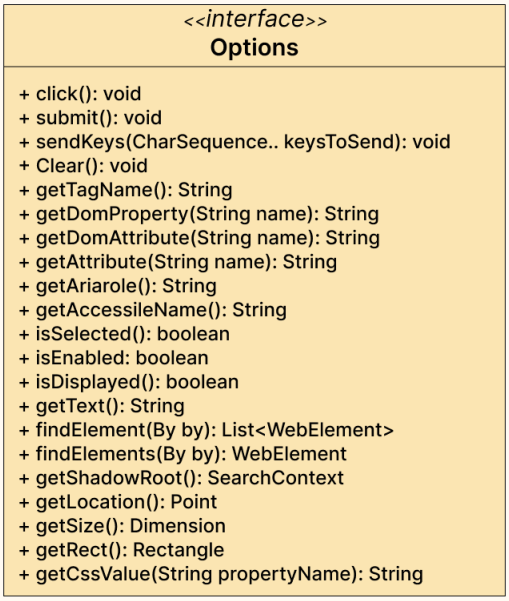
WebElement

WebElement is an interface that extends the SearchContext and TakesScreenshot interfaces.
RemoteWebElement class is the fully implemented class of the WebElement interface.





Abstract methods in WebElement Interface

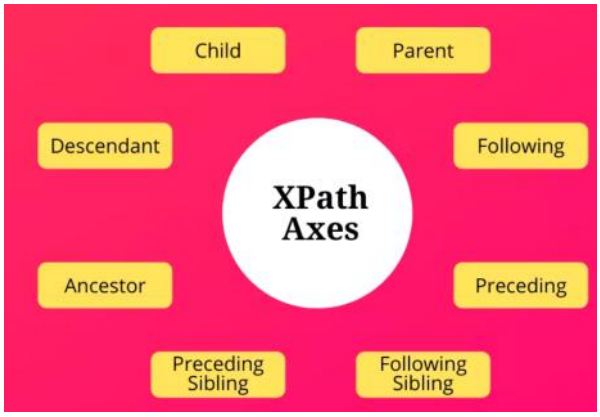


Locators

21 May 2018 13:03

Locators Supported by Selenium	1. ID 2. ClassName 3. Name 4. LinkText 5. Xpath 6. CssSelector 7. TagName 8. Partial LinkText
XPath Examples	- //input[@value='Log In']: replace input with * for wildcard - //td[@class='headertext'][@align='left'] - Child XPath: //*[@id='ctl00_mainContent_ddl_destinationStation1_CTNR'] //a[@value='AMD'] - //input[@type='submit' and @name='submitButton'] - //input[@type='submit' or @type='button']
Traversing using XPath	- Sibling to Sibling: //*[@attribute='value']/following-sibling::li[2] - Child to Parent: //*[@attribute='value']/parent::ul
Xpath: Normalize-space()	Removes Leading and Trailing Whitespace, Collapses Consecutive Whitespace //button[normalize-space(text())='Submit Form']

Xpath Union Operator	driver.get("https://www.rahulshettyacademy.com/AutomationPractice/");
	driver.manage().window().maximize();
	List<WebElement> products = driver.findElements(By.xpath("//*[@id='product']/tbody/tr"));
	for (WebElement product : products) {
	List<WebElement> cells = product.findElements(**By.xpath("./th|./td")**);
	
	Other way: //*[@id='product']/tbody/tr/*[self::td or self::tr]

Xpath Axes	XPath axes are used to specify the node selection relationship between nodes in an XML document. They define the direction in which the nodes are selected relative to the current node.  -WebElement lastInputField = driver.findElement(By.xpath("//input[last()-1]"));
Locating using Text	- //*[text()='Selenium'] - //a[contains(text(),'Samual')]
Xpath Indexing	- (//*[@class='g'])[2]
CSS Selector Examples	- input[value='Log In']: skip input for wildcard - tr[class='active'] td:nth-child(2): Accessing specific table cell using CSS Selector
Traversing using CSS	- Parent to child: parenttagname childtagname (just a space is required in CSS, whereas / is used in XPath)

Shadow Dom	Shadow DOM is a web platform feature that encapsulates a part of the DOM and hides its internal structure from the main document. This can make it challenging to locate elements using traditional Selenium locators like XPath or CSS selectors.
	
	// Access the shadow root
	JavascriptExecutor jsExecutor = (JavascriptExecutor) driver;
	WebElement shadowRoot = (WebElement) jsExecutor.executeScript
	("return document.querySelector('#shadow-host').shadowRoot");
	
	// Locate an element within the shadow root
	WebElement inputElement = shadowRoot.findElement(By.cssSelector("input#shadow-input"));
	inputElement.sendKeys("Hello, Shadow DOM!");

Using Regular Expressions	- //input[contains(@value,'Log In')]: Contains, starts-with, ends-with - //*[contains(text(),'Assyst')]

	- CSS Starts with: driver.findElement(By.cssSelector("[class^='head']")) - CSS Ends with: driver.findElement(By.cssSelector("[class$='tent']")) - CSS Contains: driver.findElements(By.cssSelector("[class*='enu']"))
Working on Console of Browser	- XPath: $x("") - CSS: $("")
Locating Items using Compound Class	- driver.findElements(By.cssSelector(".button.btn.btn-primary.btn-xs")) (button is the tagname) - driver.findElement(By.cssSelector("div[class='cb-col cb-col-100 cb-scrd-itms']"))
Select all mobiles on flipkart of less than 10000	//div[@class="Nx9bqj _4b5DiR" and number(translate(text(),",₹","")) < 10000]
getAttribute Depricated	WebElement element = driver.findElement(By.id("myElement")); DOM Property: Retrieves the value of a DOM property. Properties are dynamic and can be calculated or derived from other properties or attributes. e.g. element.getDomProperty("value"); DOM Attribute: Retrieves the value of a DOM attribute. Attributes are static and directly assigned to an element. String textContent = element.getDomProperty("readonly");

Exceptions

03 August 2021 11:34

Parent Class of All Exceptions: WebDriverException->RunTimeException->Exception->Throwable.
Catch specific exceptions, log exception details, use asserts for testing purposes, use try-catch around risky code sections.

Exception Name	Description	Resolution
NoSuchElementException	Thrown when an operation attempts to access an element that is not present on the page.	• Verify Locator • Wait for Element • Check Frame

ElementClickInterceptedException	The ElementClickInterceptedException occurs when Selenium tries to click on an element, but another element intercepts the click. This often happens when there are overlapping elements, such as pop-ups or overlays, that cover the target element	• Use Explicit Wait:
		WebDriverWait wait = new WebDriverWait(driver, 10);
		WebElement element = wait.until(ExpectedConditions.elementToBeClickable(By.id("elementId")));
		element.click();
		• Scroll to the Element:
		WebElement element = driver.findElement(By.id("elementId"));
		((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", element);
		element.click();
		• Use JavaScript Executor:
		JavascriptExecutor js = (JavascriptExecutor) driver;
		js.executeScript("arguments[0].click();", element);

ElementNotVisibleException	Thrown when an operation is attempted on an element that is present in the DOM but not visible on the page.	Scroll to the Element: Use Explicit Waits: Use JavaScript Executor:
ElementNotSelectableException	Thrown when trying to select an element that is not selectable, typically with options in a dropdown.	Use JavaScript Executor Use Explicit Waits Select Correct tag i.e option instead of div

StaleElementReferenceException	When we try to find an element using the findElement() method in WebDriver, Selenium keeps the refrenceId of that element in memory if the element is found. Later when you try to interact with the element, instead of finding it again, it gets the saved referenceId and tries to locate the element with that referenceId.
	
	Although, there can be situations where you perform some action on the page, due to which the reference of a few elements gets updated or refreshed. In such cases, you would get StaleElementReferenceException.
	
	• The referenced element is not attached to the DOM anymore.
	• The referenced element has been deleted permanently.
		Solution 1- Refresh the web page
		try {
		driver.findElement(By.cssSelector("webelement locator")).click();
		} catch(StaleElementReferenceException e) {
		driver.navigate().refresh();
		driver.findElement(By.cssSelector"webelement locator")).click();
		}
		
		Solution 2- Using PageFactory
		In POM design pattern or while using the principles of Page Factory, we locate an element using the @FindBy annotation. It helps in updating the reference of the web element each time before any action is performed on it. This becomes very useful in eliminating the occurrence of StaleElementReferenceException.

TimeoutException	Indicates that an operation did not complete within the specified timeout period specified.	
NoSuchWindowException	Thrown when switching to a window that is not available or has been closed.	
NoSuchFrameException	Raised when the WebDriver is switching to a frame that is not available or does not exist.	
NoAlertPresentException	Occurs when an attempt is made to interact with an alert, but no alert is present on the page.	
InvalidSelectorException	Indicates that the provided selector is invalid or does not match a web element.	
NoSuchSessionException	Thrown when an attempt is made to interact with a session that is no longer available.	

Waits

Wednesday, July 31, 2024 12:55 PM

Implicit Wait	• Used to set a default waiting time for the entire session. The WebDriver will wait for this duration before throwing a NoSuchElementException. Also applicable for pageLoadTimeout. - Works only with findElement and findElements. • driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));

Explicit Wait	• Allows waiting for a specific condition to be met before proceeding further.
	• Can be customized for each element or condition. Utilizes WebDriverWait and ExpectedConditions.
	
	WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
	wait.until(d->delayedText.isDisplayed());
	
	WebElement element = wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("elementId")));
	WebElement element = wait.until(ExpectedConditions.elementToBeClickable(By.id("clickableElementId")));
	WebElement element = wait.until(ExpectedConditions.presenceOfElementLocated(By.id("elementId")));
	boolean isInvisible = wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("elementId")));
	boolean isTextPresent = wait.until(ExpectedConditions.textToBePresentInElementLocated(By.id("elementId"), "expectedText"));
	boolean isAttributeValueCorrect = wait.until(ExpectedConditions.attributeToBe(By.id("elementId"), "attributeName", "expectedValue"));
	
	Warning: Do not mix implicit and explicit waits. Doing so can cause unpredictable wait times. For example, setting an implicit wait of 10 seconds and an explicit wait of 15 seconds could cause a timeout to occur after 20 seconds.

Fluent Wait(explicit WithOptions)	FluentWait in Selenium is an important class when we are dealing with AJAX elements. When a user is exploring the website on a slower network, these elements may take more time to load than what they would have taken in lab testing. To play safe, FluentWait helps us mimic such scenarios and produce the best quality web application. Wait<WebDriver> wait = new FluentWait<>(driver) .withTimeout(Duration.ofSeconds(10)) .pollingEvery(Duration.ofSeconds(2)) .ignoring(NoSuchElementException.class); WebElement element = wait.until(driver > driver.findElement(By.id("elementId"))); Selenium 4 Enhancement: Inherits from FluentWait and provides more functionality. WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10)); wait.pollingEvery(Duration.ofMillis(500)); WebElement element = wait.until(driver > driver.findElement(By.id("elementId"))); WebDriver Page 10

Interactions

21 May 2018 14:04

<table>
<tr><td>Select dropdown</td><td>Select s = new Select(driver.findElement(By.name("my_html_select_box")));
s.selectByIndex(1);

- Methods to Select Dropdown:
1. s.selectByValue(value)
2. s.selectByVisibleText(text)
3. s.selectByIndex(index)

- Methods to Deselect Dropdown:
1. s.deselectAll()
2. s.deselectByValue(value)
3. s.deselectByVisibleText(text)
4. s.deselectByIndex(index)

- Additional Dropdown Methods:
- s.isMultiple(): Returns true if the dropdown allows multiple selection.
- s.isSelected(): Returns true if the element is selected.
- s.getOptions(): Gives a list of WebElement options.
-getFirstSelectedOption()
-getAllSelectedOptions()

Without Using Select Class

// Assuming the dropdown is a <div> element with a specific class
WebElement dropdown = driver.findElement(By.cssSelector("div.dropdown-menu"));
dropdown.click();

// Assuming the option is a <a> element with a specific text
WebElement option = driver.findElement(By.xpath("//a[text()='Option 2']"));
option.click();</td></tr>
<tr><td>Alert</td><td>System.out.println(driver.switchTo().alert().getText());
driver.switchTo().alert().sendKeys("fesfe");
driver.switchTo().alert().accept(); //Accept = ok done yes
driver.switchTo().alert().dismiss();</td></tr>
<tr><td>Actions</td><td>Actions a = new Actions(driver);

<table>
<tr><td>Mouse Hover and Click</td><td>a.moveToElement(driver.findElement(By.id("nav-link-accountList"))).build().perform();</td></tr>
<tr><td>Mouse Hover, Click with Shift Key, and Typing</td><td>a.moveToElement(driver.findElement(By.id("nav-link-accountList"))).click().keyDown(Keys.SHIFT).sendKeys("hello").build().perform();</td></tr>
<tr><td>Keyboard Shortcuts</td><td>a.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();</td></tr>
<tr><td>DragandDrop</td><td>Action.clickAndHold("driver....").moveToElement("driver..").release().build().perform();</td></tr>
<tr><td>Pause</td><td>new Actions(driver)
.moveToElement(clickable)
.pause(Duration.ofSeconds(1))
.clickAndHold()
.pause(Duration.ofSeconds(1))
.sendKeys("abc")
.perform();</td></tr>
<tr><td>Release All Inputs</td><td>((RemoteWebDriver) driver).resetInputState();</td></tr>
<tr><td>sendKeysToActiveElement</td><td>@Test
public void sendKeysToActiveElement() {
driver.get("https://www.selenium.dev/selenium/web/single_text_input.html");

new Actions(driver)
.sendKeys("abc")
.perform();

WebElement textField = driver.findElement(By.id("textInput"));
Assertions.assertEquals("abc", textField.getAttribute("value"));
}</td></tr>
<tr><td>sendKeysToDesignatedElement</td><td>@Test
public void sendKeysToDesignatedElement() {
driver.get("https://www.selenium.dev/selenium/web/single_text_input.html");</td></tr>
</table>
</td></tr>
</table>

	driver.findElement(By.tagName("body")).click();
	
	WebElement textField = driver.findElement(By.id("textInput"));
	new Actions(driver)
	.sendKeys(textField, "Selenium!")
	.perform();
	
	Assertions.assertEquals("Selenium!", textField.getAttribute("value"));
	}

scrollToElement	WebElement iframe = driver.findElement(By.tagName("iframe")); new Actions(driver) .scrollToElement(iframe) .perform();
Actions vs Action	Actions is the class you use to construct a series of user interactions. Action is an interface that represents a single, completed sequence of actions built by the Actions class. Actions actions = new Actions(driver); Action combinedAction = actions .moveToElement(element1) .click() .moveToElement(element2) .contextClick() .build(); combinedAction.perform();

Window Switch	Set<String> windowHandles = driver.getWindowHandles(); List<String> windowHandlesList = new ArrayList<>(windowHandles); driver.switchTo().window(windowHandlesList.get(1));

Frames	WebElement iframe = driver.findElement(By.tagName("iframe"));
	driver.switchTo().frame(iframe);
	driver.switchTo().frame("iframe-name");
	
	switchTo().parentFrame(): In case of nested frames

Win Auth	driver.get("https://admin:admin@the-internet.herokuapp.com/");
Js Executor	JavascriptExecutor is an interface that is used to execute JavaScript with Selenium. RemoteWebDriver implements JavaScriptExe cutor. • Use getElementById() when you know the exact ID of the element. • Use querySelector() for simple CSS selectors. • Use querySelectorAll() when you need to select multiple elements. • Use getElementsByTagName() when you know the tag name of the elements. • Use getElementsByClassName() when you know the class name of the elements.

executeScript()	This method executes the test script in the context of the currently selected window or frame
executeAsyncScrip ()	This method executes the asynchronous piece of JavaScript on the current window or frame. An asynchronous script will be executed while the rest of the page continues parsing, which enhances responsiveness and application performance.

Click	JavascriptExecutor js = (JavascriptExecutor) driver;
	js.executeScript("document.getElementbyID('element id ').click();");
	
	WebElement firstYes = driver.findElement(By.xpath("//*[@id=\"yes\"]"));
	js.executeScript("arguments[0].click();", firstYes);
	
	js.executeScript("document.querySelector('#one').click();");

Scroll Down	jsExecutor.executeScript("arguments[0].scrollIntoView(true);", element);
	
	// Scroll down by 1000 pixels
	js.executeScript("window.scrollBy(0, 1000)");
	
	// Scroll up by 1000 pixels
	js.executeScript("window.scrollBy(0, -1000)");
	
	// Scroll to the bottom of the page
	js.executeScript("window.scrollTo(0, document.body.scrollHeight)");
	
	// Scroll to the top of the page
	js.executeScript("window.scrollTo(0, 0)");

Send Text	WebElement fullName = driver.findElement(By.id("fullName")); js.executeScript("document.getElementById('fullName').value='saurav';");

Get Text	String text = (String) js.executeScript("return arguments[0].value;", fullName); String text = (String) jsExecutor.executeScript("return arguments[0].innerText;", element);
Type Text	js.executeScript("document.getElementById('Email').value='SoftwareTestingMaterial.com';");

File Upload

```
driver.get("https://the-internet.herokuapp.com/upload");
    File uploadFile = new File("src/test/resources/ss.png");
    WebElement fileInput = driver.findElement(By.cssSelector("input[type=file]"));
    fileInput.sendKeys(uploadFile.getAbsolutePath());
    driver.findElement(By.id("file-submit")).click();
    WebElement fileName = driver.findElement(By.id("uploaded-files"));
    Assert.assertEquals(fileName.getText(), "ss.png");
```

File Download

```
ChromeOptions options=new ChromeOptions();
String downloadPath = Paths.get(System.getProperty("user.dir"), "downloads").toString();//creates downloads folder in the pro ject

Map<String,Object>prefs=newHashMap<>();
prefs.put("download.default_directory",downloadPath);
prefs.put("download.prompt_for_download",false);//Disabledownloadprompt
prefs.put("download.directory_upgrade",true);//Allowdirectorychanges
prefs.put("safebrowsing.enabled",true);//Enablesafebrowsing

options.setExperimentalOption("prefs",prefs);
options.setCapability(CapabilityType.ACCEPT_INSECURE_CERTS,true);

WebDriverdriver=newChromeDriver(options);
```

Window Size

```
driver.manage().window().setSize(new Dimension(1024, 768));

Dimension size = driver.manage().window().getSize();
    int width = size.getWidth();
    int height = size.getHeight();

Rectangle rectangle = elementById("property").getRect();
    System.out.println(rectangle.getDimension());
    System.out.println(rectangle.getWidth() + " " + rectangle.getHeight());


Position of Window/Element
Point position = driver.manage().window().getPosition();
    int x = position.getX();
    int y = position.getY();
    System.out.println("Window Position: X = " + x + ", Y = " + y);

 Point point = elementById("position").getLocation();
    System.out.println("X :" + point.getX());
    System.out.println("Y :" + point.getY());
```

Cookies

Add Cookie	driver.manage().addCookie(new Cookie("key", "value"));
Get Named Cookie	Cookie cookie = driver.manage().getCookieNamed("foo");

Get All Cookies	Set<Cookie> cookies = driver.manage().getCookies();
	for (Cookie cookie : cookies) {
	if (cookie.getName().equals("test1")) {
	Assertions.assertEquals(cookie.getValue(), "cookie1");
	}

Delete Cookie	driver.manage().deleteCookieNamed("test1");
Delete All Cookies	driver.manage().deleteAllCookies();

Selenium 4 Methods

21 June 2021 10:17

screenshot of web element	WebElement logo = driver.findElement(By.xpath("//div[@id='divLogo']//img")); File file = logo.getScreenshotAs(OutputType.FILE); File destFile = new File("logo.png"); FileUtils.copyFile(file, destFile);

Open a new tab	In Selenium 4, users can open a URL in a new tab along with the existing tab.
	
	For example, if the user wants to open 2 URLs in two different tabs simultaneously, it can be done with Selenium 4.
	
	driver.get("https://www.google.com/");
	driver.switchTo().newWindow(WindowType.TAB);
	driver.navigate().to("https://www.crmpro.com/");
	//switch to().window() is still required to operate on new window

Open a new window	Using Selenium 4, users can open a new window on the same browser.
	
	For example, if the user wants to access two applications in the same browser, it can now be achieved.
	
	driver.get("https://www.google.com/");
	driver.switchTo().newWindow(WindowType.WINDOW);
	driver.navigate().to("https://www.crmpro.com/");
	//switch to().window() is still required to operate on new window

Object Location	Selenium 4 enables users to obtain coordinates, dimensions, height, width, etc., as the location of web elements or objects. WebElement logo1 = driver.findElement(By.xpath("//div[@id='divLogo']//img")); System.out.println("Height: " + logo.getRect().getDimension().getHeight()); System.out.println("Width: " + logo.getRect().getDimension().getWidth()); System.out.println("X Location: " + logo.getRect().getX()); System.out.println("Y Location: " + logo.getRect().getY());
Relative Locators	These are also known as Friendly Locators, and this functionality is being added to find out the element which is present nearby to other web element or, we can say that it can find the web elements based on GUI location. There are five locators added in Selenium 4: below(): Web element located below for the specified element. toLeftOf() : Target web element which is present to the left of specified element. toRightOf(): Target web element which is presented to the right of a specified element. above(): Web element located above for the specified element. near() : Target web element which is away(approx. 50 pixels) from the specified element. sendKeys to Email Field: driver.findElement(RelativeLocator.with(By.tagName("input")).above(password)).sendKeys("ab@rmal.com"); Chaining of Relative Locators By submitLocator = RelativeLocator.with(By.tagName("button")).below(By.id("email")).toRightOf(By.id("cancel"));
Chrome Dev tools	Selenium 4 introduces changes in the Chrome DevTools API. The Chrome driver class now extends the Chromium Driver class, providing access to predefined methods for dev tool operations. Note: Chromium Driver extends the Remote Web driver class. Operations include: - Enable Network Offline - Enable Network Online - Get Console Logs - Load Insecure Website
Action Class Methods	click(WebElement): Replaces moveToElement(onElement).click(). Used for clicking on a specific web element. clickAndHold(WebElement): Replaces moveToElement(onElement).clickAndHold(). Clicks on an element without releasing the click. contextClick(WebElement): Replaces moveToElement(onElement).contextClick(). Performs the right-click operation. doubleClick(WebElement): Replaces moveToElement(element).doubleClick(). Performs a double click on an element.
ChromeOptions.addArgument	• start-maximized: Opens Chrome in maximize mode: options.addArguments("--start-maximized"); • incognito: Opens Chrome in incognito mode: options.addArguments("--incognito"); • headless: Opens Chrome in headless mode: options.addArguments("--headless"); • disable-extensions: Disables existing extensions on Chrome browser • disable-popup-blocking: Disables pop-ups displayed on Chrome browser • make-default-browser: Makes Chrome default browser • version: Prints chrome browser version

	• disable-infobars: Prevents Chrome from displaying the notification 'Chrome is being controlled by automated software • Accept TLS: options.setAcceptInsecureCerts(true);
Set Window Size	Dimension dimension = new Dimension(1024, 768); driver.manage().window().setSize(dimension);

Selenium CDP

30 August 2021 10:09

Browser Emulation

```
package com.saurav.Selenium4CDP;

import java.util.Optional;

import org.openqa.selenium.By;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.devtools.DevTools;
import org.openqa.selenium.devtools.v91.emulation.Emulation;
import org.testng.annotations.Test;

import io.github.bonigarcia.wdm.WebDriverManager;

public class MobileEmulatorTest {

	@Test
	public void testMobileEmulation() throws InterruptedException {
		WebDriverManager.chromedriver().setup();
		ChromeDriver driver=new ChromeDriver();
		driver.manage().window().maximize();
		DevTools devtools=driver.getDevTools();
		devtools.createSession();
		devtools.send(Emulation
				.setDeviceMetricsOverride(600, 1000, 50, true, Optional.empty(),
				Optional.empty(), Optional.empty(),
					Optional.empty(), Optional.empty(), Optional.empty(),
					Optional.empty(), Optional.empty(), Optional.empty()));
		driver.get("https://rahulshettyacademy.com/angularAppdemo/");
		driver.findElement(By.cssSelector(".navbar-toggler")).click();
		Thread.sleep(3000);
		driver.findElement(By.linkText("Library")).click();


	}
}
```

Custom Command

```
package com.saurav.Selenium4CDP;

import java.util.HashMap;
import java.util.Map;

import org.openqa.selenium.By;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.devtools.DevTools;
import org.testng.annotations.Test;

import io.github.bonigarcia.wdm.WebDriverManager;
```

```
public class CDPTest {

	@SuppressWarnings("unchecked")
	@Test
	public void myTest() throws InterruptedException {
	WebDriverManager.chromedriver().setup();
	ChromeDriver driver=new ChromeDriver();
	driver.manage().window().maximize();
	DevTools devtools=driver.getDevTools();
	devtools.createSession();
	Map deviceMetrics=new HashMap();
	deviceMetrics.put("width", 600);
	deviceMetrics.put("height", 1000);
	deviceMetrics.put("deviceScaleFactor", 50);
	deviceMetrics.put("mobile", true);
	driver.executeCdpCommand("Emulation.setDeviceMetricsOverride", deviceMetrics);
	driver.get("https://rahulshettyacademy.com/angularAppdemo/");
	driver.findElement(By.cssSelector(".navbar-toggler")).click();
	Thread.sleep(3000);
	driver.findElement(By.linkText("Library")).click();
	}
}
```

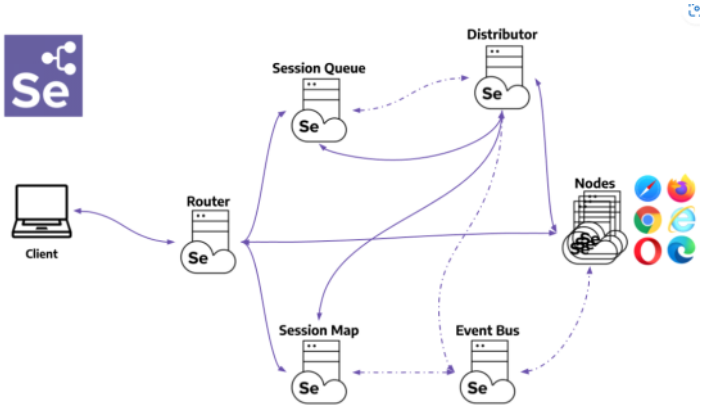
Geo Location Override

```
package com.saurav.Selenium4CDP;

import java.util.HashMap;
import java.util.Map;

import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.devtools.DevTools;
import org.testng.annotations.Test;

import io.github.bonigarcia.wdm.WebDriverManager;

public class GeoLocationTest {

	@Test
	public void geoTest() {
		WebDriverManager.chromedriver().setup();
		ChromeDriver driver=new ChromeDriver();
		driver.manage().window().maximize();
		DevTools devtools=driver.getDevTools();
		devtools.createSession();
		Map<String,Object> geoLocale=new HashMap<>();
		geoLocale.put("latitude",35.652832);
		geoLocale.put("longitude", 139.839478);
		geoLocale.put("accuracy", 100);
		driver.executeCdpCommand("Emulation.setGeolocationOverride",geoLocale);
		driver.get("https://www.google.com");
		driver.navigate().refresh();
	}
}
```

Grid

03 April 2024 19:40

Hub	• The hub is the central point of control in Selenium Grid. It's typically located on your local machine where you write and trigger your Selenium tests. • The hub receives test requests and then decides which node (or machine) will execute the test based on the requirements specified in the test (e.g., browser type, operating system). • The hub can manage multiple nodes, directing test execution to the appropriate machine. • A Hub is composed by the following components: Router, Distributor, Session Map, New Session Queue, and Event Bus.
Router	The Router is the entry point of the Grid, receiving all external requests, and forwards them to the correct component. If the Router receives a new session request, it will be forwarded to the New Session Queue. If the request belongs to an existing session, the Router will query the Session Map to get the Node ID where the session is running, and then the request will be forwarded directly to the Node. The Router balances the load in the Grid by sending the requests to the component that is able to handle them better, without overloading any component that is not needed in the process.
Distributor	The Distributor has two main responsibilities: • Register and keep track of all Nodes and their capabilities: A Node registers to the Distributor by sending a Node registration event through the Event Bus. The Distributor reads it, and then tries to reach the Node via HTTP to confirm its existence. If the request is successful, the Distributor registers the Node and keeps track of all Nodes capabilities through the GridModel. • Query the New Session Queue and process any pending new session requests: When a new session request is sent to the Router, it gets forwarded to the New Session Queue, where it will wait in the queue. The Distributor will poll the New Session Queue for pending new session requests, and then finds a suitable Node where the session can be created. After the session has been created, the Distributor stores in the Session Map the relation between the session id and Node where the session is being executed
Session Map	The Session Map is a data store that keeps the relationship between the session id and the Node where the session is running. It supports the Router in the process of forwarding a request to the Node. The Router will ask the Session Map for the Node associated to a session id
New Session Queue	T he New Session Queue holds all the new session requests in a FIFO order. It has configurable parameters for setting the request timeout and request retry interval (how often the timeout will be checked). The Router adds the new session request to the New Session Queue and waits for the response. The New Session Queue regularly checks if any request in the queue has timed out, if so the request is rejected and removed immediately. The Distributor regularly checks if a slot is available. If so, the Distributor polls the New Session Queue for the first matching request. The Distributor then attempts to create a new session. Once the requested capabilities match the capabilities of any of the free Node slots, the Distributor attempts to get the available slot. If all the slots are busy, the Distributor will send the request back to the queue. If request times out while retrying or adding to the front of the queue, it will be rejected. After a session is created successfully, the Distributor sends the session information to the New Session Queue, which then gets sent back to the Router, and finally to the client.
Nodes	A Grid can contain multiple Nodes. Each Node manages the slots for the available browsers of the machine where it is running. The Node registers itself to the Distributor through the Event Bus, and its configuration is sent as part of the registration message. By default, the Node auto-registers all browser drivers available on the path of the machine where it runs. It also creates one slot per available CPU for Chromium based browsers and Firefox. For Safari, only one slot is created. Through a specific configuration, it can run sessions in Docker containers or relay commands. A Node only executes the received commands, it does not evaluate, make judgments, or control anything other than the flow of commands and responses. The machines where the Node is running does not need to have the same operating system as the other components. For example, A Windows Node might have the capability of offering IE Mode on Edge as a browser option, whereas this would not be possible on Linux or Mac, and a Grid can have multiple Nodes configured with Windows, Mac, or Linux
Event Bus	The Event Bus serves as a communication path between the Nodes, Distributor, New Session Queue, and Session Map. The Grid does most of its internal communication through messages, avoiding expensive HTTP calls. When starting the Grid in its fully distributed mode, the Event Bus is the first component that should be started.
Diagram	Hub recieves user request of port 4444 Hub recieves messages from node on 4442 Hub sends messages to browser on 4443

docker-compose-hub.yml	x-common-node: &common-node
	  shm_size: '2gb'
	  depends_on:
	  - selenium-hub
	  environment:
	  - SE_EVENT_BUS_HOST=selenium-hub
	  - SE_EVENT_BUS_PUBLISH_PORT=4442
	  - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
	  #- SE_VNC_NO_PASSWORD=1
	
	services:
	  selenium-hub:
	  image: selenium/hub:latest
	  container_name: selenium-hub
	  ports:
	  - "4444:4444"
	  - "4442:4442"
	  - "4443:4443"
	  networks:
	  - selenium-network
	
	  chrome:
	  image: selenium/node-chrome:latest
	  <<: *common-node
	  environment:
	  - SE_NODE_OVERRIDE_MAX_SESSIONS=true
	  - SE_NODE_MAX_SESSIONS=2
	  networks:
	  - selenium-network
	
	  edge:
	  image: selenium/node-edge:latest
	  <<: *common-node
	  networks:
	  - selenium-network
	
	  firefox:
	  image: selenium/node-firefox:latest
	  <<: *common-node
	  networks:
	  - selenium-network
	
	networks:
	  selenium-network:
	  driver: bridge

docker-compose-node.yml	services:
	  chrome:
	  image: selenium/node-chrome:latest
	  shm_size: '2gb'
	  environment:
	  - SE_EVENT_BUS_HOST=192.168.1.22 # IP address of the hub from the first compose file. #docker network inspect grid_selenium -network
	  - SE_EVENT_BUS_PUBLISH_PORT=4442
	  - SE_EVENT_BUS_SUBSCRIBE_PORT=4443
	  - SE_NODE_OVERRIDE_MAX_SESSIONS=true
	  - SE_NODE_MAX_SESSIONS=2
	  #- SE_VNC_NO_PASSWORD=1
	  networks:
	  - selenium-network

	networks:
	  selenium-network:
	  driver: bridge

Starting Grid	docker-compose -f docker-compose.yml up -d docker-compose -f docker-compose-node.yml down Access hub UI at localhost:4444
Control number of browser instances from command line	docker-compose up --scale chrome=3 --scale firefox=2 -d

Code	WebDriver driver = null;
	  try {
	  URL gridUrl = new URI("http://192.168.1.22:4444/wd/hub").toURL();
	  DesiredCapabilities capabilities = new DesiredCapabilities();
	  capabilities.setCapability("browserName", "MicrosoftEdge");
	  driver = new RemoteWebDriver(gridUrl, capabilities);
	  driver.get("https://www.google.com");
	  String title = driver.getTitle();
	  System.out.println("Page title is: " + title);
	  } catch (Exception e) {
	  System.out.println(e.getMessage());
	  } finally {
	  if (driver != null) {
	  driver.quit();
	  }
	  }

java -jar selenium-server-4.27.0.jar node --hub http://192.168.1.16:4444 --host 192.168.1.13 --port 5555

Thread Local

16 November 2021 16:56

ThreadLocal in Java is another way to achieve thread-safety. Java ThreadLocal class provides thread-local variables. It enables you to create variables that can only be read and write by the same thread. If two threads are executing the same code and that code has a reference to a ThreadLocal variable then the two threads can't see the local variable of each other.

```
// ThreadLocal to ensure each thread has its own WebDriver instance
private static final ThreadLocal<WebDriver> tlDriver = ThreadLocal.withInitial(() -> null);

public static void initDriver(String browser) {
    if (tlDriver.get() == null) {
        WebDriver driver = switch (browser.toLowerCase()) {
            case "chrome" -> new ChromeDriver();
            case "firefox" -> new FirefoxDriver();
            case "edge" -> new EdgeDriver();
            default -> throw new IllegalArgumentException("Unsupported browser: " + browser);
        };
        tlDriver.set(driver);
    }
}

public static WebDriver getDriver() {
    WebDriver driver = tlDriver.get();
    if (driver == null) {
        throw new IllegalStateException("Driver is not initialized. Call initDriver() first.");
    }
    return driver;
}

public static void quitBrowser() {
    WebDriver driver = tlDriver.get();
    if (driver != null) {
        driver.quit();
        tlDriver.remove();
    }
}
```

BNY

23 January 2022 09:58

```
public boolean waitForTheElementToAppear(String locator, int time,WebDriver driver) throws
InterruptedException {
		boolean isElementFound = false;
		for (int i = 0; i < time; i++) {
			if (isElementPresent(locator, driver)) {
				isElementFound = true;
				break;
			}
			Thread.sleep(1000);
		}
		return isElementFound;
	}

	public boolean isElementPresent(String locator, WebDriver driver) {
		boolean isElementFound = false;
		try {
			waitForTheElementToAppear(locator,10, driver);
			if (driver.findElements(By.xpath(locator)).size() > 0) {
				isElementFound = true;
			}
		} catch (Exception e) {
			e.printStackTrace();
		}
		return isElementFound;
	}

	public boolean isElementDisplayed(String locator, WebDriver driver) {
		boolean isElementFound = false;
		try {
			waitForTheElementToAppear(locator,10, driver);
			if (driver.findElements(By.xpath(locator)).size() > 0) {
				isElementFound = driver.findElement(By.xpath(locator)).isDisplayed();
			}
		} catch (Exception e) {
			e.printStackTrace();
		}
		return isElementFound;
	}
```

Selvaraju, Manivannan 16:14

Here is the alternative solution for Identifying Xpath of disappearing elements

1. Inspect the element which will display the disappearing elements

2. Right click on the element tag and select 'Break on' and select 'attribute modifications'. Once you selected them, you will be able to see it under DOM Breakpoints section.

3. Now click on the element which will display the disappearing elements. Debugger mode will open.

4. Now press F8, until you are able to inspect the disappearing element.

Please refer the mail for screenshots

16:27

Locating disappearing elements

Xpath when we want to iterate by updating numbers in xpath

```
private String tradeGridQty1 = "//div[@class='ag-pinned-left-cols-container']/div[@row-index='%s']/div[@col-id='strOrdQty']";

public String getElementText(String ele,int num) {
    WebElement element = cdriver.findElement(By.xpath(String.format(ele, String.valueOf(num))));
    String actualtext = element.getText();
    return actualtext;
}
System.out.println("total orders: " + rowCount);
for (int k = 0; k < rowCount; k++) {
    leftPanelStatus = getPageObjectManager().getCommonPage().getElementText(tradeGridLeftPanStatus, k);
    transType = getPageObjectManager().getCommonPage().getElementText(tradeGridTransType, k);
    SecuritySymbol = getPageObjectManager().getCommonPage().getElementText(tradeGridSecSymbol, k);
    qty = getPageObjectManager().getCommonPage().getElementText(tradeGridQty1, k);
    date1 = getPageObjectManager().getCommonPage().getElementText(tradeGridDate, k);
    String qty1 = qty.replaceAll("[^\\d.]+", "");
    if (tradeSharesDollars.equalsIgnoreCase("Shares")) {
```

Validate is field disabled

```
public void isFieldDisabled(String fieldName) {
    boolean isElementDisabled;
    if(fieldName.equals("Call / Put"))
    isElementDisabled = commonActions
            .getAttributeValue(By.xpath("(//label[contains(text(),'"+fieldName+"')])[2]/parent::div/mat-form-field"),
                    "class")
            .trim().contains("mat-form-field-disabled");
    else
        isElementDisabled = commonActions
                .getAttributeValue(By.xpath("//label[contains(text(),'"+fieldName+"')]/parent::div/mat-form-field"),
                "class")
        .trim().contains("mat-form-field-disabled");

    if (isElementDisabled) {
        Screenshots.addStepInReport(true, fieldName +" field is displayed and disabled");
    } else {
        System.out.println(fieldName +" field is enabled");
        AssertionLibrary.assertTrue(false, fieldName +" field is enabled", Screenshot.REQUIRED);
    }
  }
}
```

```
public String getAttributeValue(String locator, String attributeName) {
    String text = null;
    try {
        WebElement localWebElement = getElement(By.xpath(locator));
        text = localWebElement.getAttribute(attributeName);
    } catch (Exception e) {
        e.printStackTrace();
    }
    return text;
}
```

Design Patterns

30 September 2024 08:25

Singleton

- Create Private Static Instance of the Class
- Create Private Constructor
- Public Static getInstance for instance/object

```
public class Browser {  4 usages  new *

    private static Browser browser;  3 usages

    private Browser(){}  1 usage  new *

    public static Browser getBrowser(){  1 usage  new *
        if(browser==null){
            browser=new Browser();
        }
        return browser;
    }

    public void display(){  no usages  new *
        System.out.println("Browser Info");
    }

}
```

```
public class Main {
   public static void main(String[] args) {
      Browser.getBrowser().display();
   }
}
```

Singleton pattern does not work well in parallel execution: Use ThreadLocal with Parallel Execution

Selenium Singleton

```
public class WebDriverSingleton {
  private static final ThreadLocal<WebDriver> driver = new ThreadLocal<>();
  private WebDriverSingleton() {}
  public static WebDriver getDriver(String browser) {
    if (driver.get() == null) {
      switch (browser.toLowerCase()) {
        case "chrome":
          ChromeOptions chromeOptions = new ChromeOptions();
          chromeOptions.addArguments("--start-maximized");
          driver.set(new ChromeDriver(chromeOptions));
          break;
        case "firefox":
          FirefoxOptions firefoxOptions = new FirefoxOptions();
          firefoxOptions.addArguments("--start-maximized");
          driver.set(new FirefoxDriver(firefoxOptions));
          break;
        case "edge":
          EdgeOptions edgeOptions = new EdgeOptions();
          edgeOptions.addArguments("--start-maximized");
          driver.set(new EdgeDriver(edgeOptions));
          break;
        default:
          throw new IllegalArgumentException("Unsupported browser: " + browser);
      }
    }
    return driver.get();
  }

  public static void quitDriver() {
    if (driver.get() != null) {
      driver.get().quit();
      driver.remove();
    }
  }
}
```

```
@BeforeMethod
   @Parameters("browser")
   public void setup(String browser) {
      driver = WebDriverSingleton.getDriver(browser);
   }

   @Test
   public void testExample1() {
      driver.get("https://www.example.com");
      Assert.assertEquals(driver.getTitle(), "Example Domain");
   }
```

Factory Pattern(Used by webDriver)

A Factory Pattern or Factory Method Pattern(Follows LSP) says that just define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate. In other words, subclasses are responsible to create the instance of the cl ass.

The Factory Method pattern is a creational design pattern that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created. It encapsulates the object creation logic, decoupling it from t he client code that uses the objects.

The Factory Method Pattern is also known as Virtual Constructor.

Advantages

- The Factory Pattern allows WebDriver to abstract away the specific implementation details of different browser drivers.
- This makes it easier to switch between browsers without modifying client code.
- New browser drivers can be added to the framework without affecting existing tests.
- Different browsers can be configured independently.

PageFactory.initElements(driver, this) is used to initialize the page objects' elements. Selenium uses reflection to instanti ate and initialize the elements of the page, which is another form of the Factory pattern.

Components of Factory Method Design Pattern

- Product: Represents the interface of the objects created by the factory method.
- ConcreteProduct: Implements the Product interface and represents the concrete objects created by the factory method.
- Creator: Declares the factory method, which returns an instance of the Product interface.
- ConcreteCreator: Implements the factory method to create instances of ConcreteProduct.

```
public interface BrowserDriver {
   WebDriver createDriver();
}

public class ChromeDriverManager implements BrowserDriver {
   @Override
   public WebDriver createDriver() {
      return new ChromeDriver();
   }
}

public class EdgeDriverManager implements BrowserDriver {
   @Override
   public WebDriver createDriver() {
      return new EdgeDriver();
   }
}

public class DriverFactory {
   public static BrowserDriver getDriver(String browserType) {
      return switch (browserType.toLowerCase()) {
         case "chrome" -> new ChromeDriverManager();
         case "firefox" -> new FirefoxDriverManager();
         case "edge" -> new EdgeDriverManager();
         default -> throw new IllegalArgumentException("Unknown browser type: " + browserType);
      };
   }
}

  public static void main(String[] args) {
      String browserType = "edge";
      WebDriver driver = DriverFactory.getDriver(browserType).createDriver();
      driver.manage().window().maximize();
      driver.get("https://www.google.com/");
      System.out.println(driver.getTitle());
      driver.quit();
   }
```

	The Factory Method Design Pattern is a powerful tool for creating objects in a flexible, decoupled, and extensible manner. By encapsulating object creation logic in subclasses, the Factory Method pattern promotes code reusability, maintainability, and scalability. While it may introduce some complexity, especially in large codebases, its benefits outweigh the drawbacks, making it a valua ble addition to any developer's toolkit. **When PageFactory.initElements is called:** • It reads all the fields annotated with @FindBy (or similar annotations). • For each field, it dynamically generates a proxy object (using a mechanism like Java's Reflection or Proxy classes). • The proxy object doesn't immediately locate the WebElement; instead, it defers the lookup until the element is actually interacted with. This is known as lazy initialization. • The proxy handles the interaction with the WebDriver, locating the element on the page at runtime.

Builder Pattern	The Builder Design Pattern is a creational pattern that allows for the step-by-step creation of complex objects. It is mostly used when object can't be created in single step like in the de-serialization of a complex object
	
	Advantages
	• Useful when the constructor has many fields
	• Userful when the constructor has few optional and few mandatory fields
	• Can be used in UI automation for fill registration forms
	
	@Data
	public class Product {
	private String name;
	private Double price;
	//optional fields
	private String description;
	private String category;
	private Double discount;
	private boolean freeshipping;
	private int stock;
	
	private Product(ProductBuilder builder) {
	this.name = builder.name;
	this.price = builder.price;
	this.description = builder.description;
	this.category = builder.category;
	this.discount = builder.discount;
	this.freeshipping = builder.freeshipping;
	this.stock = builder.stock;
	}
	
	public static class ProductBuilder {
	private String name;
	private Double price;
	//optional fields
	private String description;
	private String category;
	private Double discount;
	private boolean freeshipping;
	private int stock;
	
	//public constructor for mandatory fields
	public ProductBuilder(String name, Double price) {
	this.name = name;
	this.price = price;
	}
	//setter for optional fields
	public ProductBuilder setDescription(String description) {
	this.description = description;
	return this;
	}
	
	public ProductBuilder setCategory(String category) {
	this.category = category;
	return this;
	}
	
	public ProductBuilder setDiscount(Double discount) {
	this.discount = discount;
	return this;
	}
	
	public ProductBuilder setFreeshipping(boolean freeshipping) {
	this.freeshipping = freeshipping;
	return this;
	}

```
    public ProductBuilder setStock(int stock) {
        this.stock = stock;
        return this;
    }

    public Product build(){
        return new Product(this);
    }
  }
}



public class Runner {
  public static void main(String[] args) {
    Product myTv = new Product.ProductBuilder("MI TV", 12500.0).build();
    System.out.println(myTv);

    Product appleTv = new Product.ProductBuilder("Apple TV", 125000.0)
          .setDescription("Mehenga TV")
          .setCategory("Electronics")
          .build();
    appleTv.setDiscount(5.2);
    System.out.println(appleTv);
  }
}
```

Strategy Pattern

Strategy Design Pattern is a behavioral design pattern that allows the behavior of a class instance to be changed dynamically .
This pattern is useful when there are multiple ways or strategies to perform a task, and you want to switch between these str ategies at runtime. Imagine a scenario where we are developing test scripts to validate API responses. For this task, we can employ mul tiple strategies, such as validating for a valid response or checking for an invalid response.

Designing the Strategy Design pattern has 4 steps

Step 1: Define the Strategy Interface	`@FunctionalInterface`
	`public interface ValidationStrategy {`
	`  void validate(Response response);`
	`}`

Step 2: Implement Concrete Strategy Classes	`import java.util.Set;`
	
	`public class ValidResponseStrategy implements ValidationStrategy {`
	`  @Override`
	`  public void validate(Response response) {`
	`  int expectedStatusCode = 200;`
	`  if (response.getStatusCode() != expectedStatusCode) {`
	`  throw new AssertionError(`
	`  "Expected status code " + expectedStatusCode + ", but got " + response.getStatusCode()`
	`  );`
	`  }`
	`  }`
	`}`
	
	`public class InvalidResponseStrategy implements ValidationStrategy {`
	`  private static final Set<Integer> INVALID_STATUS_CODES = Set.of(400, 404, 500);`
	
	`  @Override`
	`  public void validate(Response response) {`
	`  if (INVALID_STATUS_CODES.contains(response.getStatusCode())) {`
	`  throw new AssertionError(`
	`  "Did not expect status code " + response.getStatusCode() + ", which is considered invalid"`
	`  );`
	`  }`
	`  }`
	`}`

Step 3: Create the Context Class	`public class ResponseValidatorContext {`
	`  private ValidationStrategy strategy;`
	
	`  public ResponseValidatorContext(ValidationStrategy strategy) {`
	`  this.strategy = strategy;`
	`  }`
	
	`  public void setStrategy(ValidationStrategy strategy) {`
	`  this.strategy = strategy;`
	`  }`
	
	`  public void validateResponse(Response response) {`

		strategy.validate(response);
		}
		}

	Step 4: Utilize the Strategy Pattern	public class Main {
		public static void main(String[] args) {
		Response response = new Response(200);
		
		// Using ValidResponseStrategy with a lambda expression
		ValidationStrategy validStrategy = r -> {
		int expectedStatusCode = 200;
		if (r.getStatusCode() != expectedStatusCode) {
		throw new AssertionError(
		"Expected status code " + expectedStatusCode + ", but got " + r.getStatusCode()
		);
		}
		};
		
		// Using InvalidResponseStrategy with a lambda expression
		Set<Integer> invalidStatusCodes = Set.of(400, 404, 500);
		ValidationStrategy invalidStrategy = r -> {
		if (invalidStatusCodes.contains(r.getStatusCode())) {
		throw new AssertionError(
		"Did not expect status code " + r.getStatusCode() + ", which is considered invalid"
		);
		}
		};
		
		ResponseValidatorContext context = new ResponseValidatorContext(validStrategy);
		context.validateResponse(response);
		
		context.setStrategy(invalidStrategy);
		context.validateResponse(response);
		}
		}

The Strategy design pattern is a powerful tool in the arsenal of software engineers aiming to write clean, flexible, and main tainable code. Particularly useful in scenarios involving varying algorithms, it separates the concerns of algorithm selection from the cont ext in which they are used. By embracing this pattern, developers can ensure that their code is both scalable and resilient to change, emb odying the principles of good software design.

Decorator Design Pattern(OCP)

Structural design pattern used to dynamically add new functionality to an object without altering its structure. It achieves this by wrapping the original object with a set of decorator objects that enhance or modify its behavior.

- Base Component: This is the interface or abstract class representing the core entity (e.g., Coffee).
- Concrete Component: This is the base implementation of the component (e.g., PlainCoffee).
- Decorator: This is an abstract class or interface implementing the base component, which adds the ability to decorate.
- Concrete Decorators: These are the actual decorators that add functionality (e.g., Milk, Sugar).

```
// Base Component
interface Coffee {
   String getDescription();
   double getCost();
}

// Concrete Component
class PlainCoffee implements Coffee {
   @Override
   public String getDescription() {
      return "Plain Coffee";
   }

   @Override
   public double getCost() {
      return 5.0; // Base price of plain coffee
   }
}

// Decorator
abstract class CoffeeDecorator implements Coffee {
   protected Coffee coffee;

   public CoffeeDecorator(Coffee coffee) {
      this.coffee = coffee;
   }

   @Override
```

```
    public String getDescription() {
        return coffee.getDescription();
    }

    @Override
    public double getCost() {
        return coffee.getCost();
    }
}

// Concrete Decorators
class Milk extends CoffeeDecorator {
    public Milk(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", Milk";
    }

    @Override
    public double getCost() {
        return super.getCost() + 1.5; // Additional cost for milk
    }
}

class Sugar extends CoffeeDecorator {
    public Sugar(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", Sugar";
    }

    @Override
    public double getCost() {
        return super.getCost() + 0.5; // Additional cost for sugar
    }
}

class WhippedCream extends CoffeeDecorator {
    public WhippedCream(Coffee coffee) {
        super(coffee);
    }

    @Override
    public String getDescription() {
        return super.getDescription() + ", Whipped Cream";
    }

    @Override
    public double getCost() {
        return super.getCost() + 2.0; // Additional cost for whipped cream
    }
}

// Main Class
public class CoffeeShop {
    public static void main(String[] args) {
        // Start with a plain coffee
        Coffee myCoffee = new PlainCoffee();
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());

        // Add Milk
        myCoffee = new Milk(myCoffee);
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());

        // Add Sugar
        myCoffee = new Sugar(myCoffee);
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());

        // Add Whipped Cream
        myCoffee = new WhippedCream(myCoffee);
```

```
        System.out.println(myCoffee.getDescription() + " -> $" + myCoffee.getCost());
    }
}
```

Selenium Decorator

```
// Base Component
interface Element {
    void click();
    void sendKeys(String text);
    String getText();
}

// Concrete Component
class BaseElement implements Element {
    private WebElement webElement;

    public BaseElement(WebElement webElement) {
        this.webElement = webElement;
    }

    @Override
    public void click() {
        webElement.click();
    }

    @Override
    public void sendKeys(String text) {
        webElement.sendKeys(text);
    }

    @Override
    public String getText() {
        return webElement.getText();
    }
}

// Decorator
abstract class ElementDecorator implements Element {
    protected Element element;

    public ElementDecorator(Element element) {
        this.element = element;
    }

    @Override
    public void click() {
        element.click();
    }

    @Override
    public void sendKeys(String text) {
        element.sendKeys(text);
    }

    @Override
    public String getText() {
        return element.getText();
    }
}

// Concrete Decorators
class LoggingElement extends ElementDecorator {
    public LoggingElement(Element element) {
        super(element);
    }

    @Override
    public void click() {
        System.out.println("Clicking element");
        super.click();
    }

    @Override
    public void sendKeys(String text) {
        System.out.println("Typing: " + text);
        super.sendKeys(text);
    }
```

```
    @Override
    public String getText() {
        String text = super.getText();
        System.out.println("Reading text: " + text);
        return text;
    }
}

class ExceptionHandlingElement extends ElementDecorator {
    public ExceptionHandlingElement(Element element) {
        super(element);
    }

    @Override
    public void click() {
        try {
            super.click();
        } catch (Exception e) {
            System.err.println("Error clicking element: " + e.getMessage());
        }
    }

    @Override
    public void sendKeys(String text) {
        try {
            super.sendKeys(text);
        } catch (Exception e) {
            System.err.println("Error typing: " + e.getMessage());
        }
    }
}
```

Usage in Framework:

```
WebElement webElement = driver.findElement(By.id("example"));

// Wrap with decorators
Element element = new LoggingElement(new ExceptionHandlingElement(new BaseElement(webElement)));

// Perform actions
element.click();
element.sendKeys("Test");
System.out.println(element.getText());
```

Use Case 2: Screenshot on Failure

```
class ScreenshotElement extends ElementDecorator {
    private WebDriver driver;

    public ScreenshotElement(Element element, WebDriver driver) {
        super(element);
        this.driver = driver;
    }

    @Override
    public void click() {
        try {
            super.click();
        } catch (Exception e) {
            takeScreenshot();
            throw e;
        }
    }

    @Override
    public void sendKeys(String text) {
        try {
            super.sendKeys(text);
        } catch (Exception e) {
            takeScreenshot();
            throw e;
```

```
        }
    }

    private void takeScreenshot() {
        File screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
        try {
            FileUtils.copyFile(screenshot, new File("screenshot_" + System.currentTimeMillis() + ".png"));
            System.out.println("Screenshot captured!");
        } catch (IOException e) {
            System.err.println("Failed to save screenshot: " + e.getMessage());
        }
    }
}

Usage:
Element element = new ScreenshotElement(new BaseElement(webElement), driver);
element.click();
```

Apache POI

15 December 2024 00:00

Read Data	public static Map<String, String> getRowData(String filePath, int rowNumber) throws IOException {
	Map<String, String> rowData = new HashMap<>();
	
	try (FileInputStream fileInputStream = new FileInputStream(filePath); Workbook workbook = new XSSFWorkbook(fileInputStream)) {
	Sheet sheet = workbook.getSheetAt(0);
	Row headerRow = sheet.getRow(0);
	if (headerRow == null) {
	throw new IllegalArgumentException("Header row is missing in the sheet.");
	}
	Row dataRow = sheet.getRow(rowNumber);
	if (dataRow == null) {
	throw new IllegalArgumentException("Row number " + rowNumber + " is missing in the sheet.");
	}
	for (Cell headerCell : headerRow) {
	int columnIndex = headerCell.getColumnIndex();
	String header = headerCell.getStringCellValue();
	Cell dataCell = dataRow.getCell(columnIndex);
	String value = getCellValueAsString(dataCell);
	rowData.put(header, value);
	}
	}
	return rowData;
	}
	
	private static String getCellValueAsString(Cell cell) {
	return switch (cell.getCellType()) {
	case STRING -> cell.getStringCellValue();
	case NUMERIC -> String.valueOf(cell.getNumericCellValue());
	case BOOLEAN -> String.valueOf(cell.getBooleanCellValue());
	default -> "";
	};
	}

Update Data	public static void updateCellData(String filePath, int rowNumber, String columnHeader, String newValue) throws IOException {
	try (FileInputStream fileInputStream = new FileInputStream(filePath); Workbook workbook = new XSSFWorkbook(fileInputStream)) {
	Sheet sheet = workbook.getSheetAt(0);
	Row headerRow = sheet.getRow(0);
	if (headerRow == null) {
	throw new IllegalArgumentException("Header row is missing in the sheet.");
	}
	int columnIndex = -1;
	for (Cell headerCell : headerRow) {
	if (headerCell.getStringCellValue().equalsIgnoreCase(columnHeader)) {
	columnIndex = headerCell.getColumnIndex();
	break;
	}
	}
	if (columnIndex == -1) {
	throw new IllegalArgumentException("Column header \"" + columnHeader + "\" not found in the sheet.");
	}
	Row dataRow = sheet.getRow(rowNumber);
	if (dataRow == null) {
	dataRow = sheet.createRow(rowNumber);
	}
	Cell cellToUpdate = dataRow.getCell(columnIndex);
	if (cellToUpdate == null) {
	cellToUpdate = dataRow.createCell(columnIndex);
	}
	cellToUpdate.setCellValue(newValue);
	try (FileOutputStream fileOutputStream = new FileOutputStream(filePath)) {
	workbook.write(fileOutputStream);
	}
	}
	}

Miscellaneous

21 May 2018 14:28

Right-click using WebDriver	Actions act = new Actions(driver); act.contextClick().perform();

Drag and drop using WebDriver	Actions act = new Actions(driver);
	WebElement source = driver.findElement(By.xpath(" -----")); // source element
	WebElement target = driver.findElement(By.xpath(" -----")); // target element
	act.dragAndDrop(source, target).perform();

Verify checkbox/radio state	boolean isChecked = element.isSelected();

Simulate browser back and forward	driver.navigate().back();
	driver.navigate().forward();
	driver.navigate().refresh();

Set Dimension Of Browser	Dimension d = new Dimension(420,600); driver.manage().window().setSize(d);

Double Click	Actions action = new Actions(driver);
	action.doubleClick(ele);
	action.perform();

Sending Keys to Element	Keyboard Keys with WebDriver: driver.findElement(By...).sendKeys(Keys.TAB);
isDisplayed()	to check visibility (only for hidden items in page source). To check if an element is present or not, use size() not isDisplayed(). isSelected(): Checks if the element is selected (e.g., checkboxes). isDisplayed(): Verifies if the element is present in the DOM and displayed on the web page. isEnabled(): Checks if the element is currently enabled for user interaction. Mostly textbox might be disabled. This function checks for disabled attribute to be true.
In headless more better to manually scroll to element else test might fail in some cases.	JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("window.scrollBy(0,500)");
Scroll Inside Table	js.executeScript("document.querySelector(.tableFixHead').scrollTop=5000");(scrollLeft is also a method)

Start Chrome with Extention	ChromeOptions options = new ChromeOptions();
	options.addExtensions(new File("Path to CRX File"));
	DesiredCapabilities capabilities = new DesiredCapabilities();
	capabilities.setCapability(ChromeOptions.CAPABILITY, options);
	ChromeDriver driver = new ChromeDriver(capabilities);

Block PopUps	ChromeOptions options = new ChromeOptions(); options.setExperimentalOption("excludeSwitches", Arrays.asList("disable-popup-blocking"));

Taking Screenshot	File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
	// Copy the screenshot to a destination file
	FileUtils.copyFile(src, new File("C:\\Users\\singh\\Desktop\\screenshot.png"));

Carry Xpath	public void selectValuefromDropdown(String dropdownLocator, String optionsLocator, String option) { clickElement (dropdownLocator, "Dropdown Clicked"); List<WebElement> dates = driver.findElements(By.xpath(optionsLocator)); dates.stream().filter(i > i.findElement(**By.xpath("./span")**).getText().equalsIgnoreCase(option)).limit(1).forEach(i -> i.click()); }

Control Clicks	for (int i = 1; i < count; i++) {
	String tabclick = Keys.chord(Keys.CONTROL, Keys.ENTER);
	scope.findElements(By.tagName("a")).get(i).sendKeys(tabclick);
	
	}

Limit Scope	WebElement footer= driver.findElement.... System.out.println(Footer.findElements(By.tagName("a")).size()); Using String.format: String xpath = String.format("//button[@id='%s']", buttonId);

Broken Link Validator	public static void verifyLink(String url) {
	try {
	URL link = new URL(url);
	HttpURLConnection httpURLConnection = (HttpURLConnection) link.openConnection();
	httpURLConnection.setConnectTimeout(3000); // Set connection timeout to 3 seconds
	httpURLConnection.connect();
	
	if (httpURLConnection.getResponseCode() == 200) {
	System.out.println(url + " - " + httpURLConnection.getResponseMessage());
	} else {
	System.out.println(url + " - " + httpURLConnection.getResponseMessage() + " - " + "is a broken link");
	}
	} catch (Exception e) {
	System.out.println(url + " - " + "is a broken link");

<table>
<tr><td></td><td>}
}</td></tr>
<tr><td>Page Factory</td><td>Structure: Similar to POM, but with a focus on initializing web elements at the time of page class creation. PageFactory is used to Initialize Elements of a Page class without having to use ‘FindElement’ or ‘FindElements’.

Initialization: Elements are initialized using the PageFactory.initElements() method, which is simpler and reduces the boilerplate code.

@FindBy(id = "username")
private WebElement usernameField;

public LoginPage(WebDriver driver) {
PageFactory.initElements(driver, this);
}

public void enterUsername(String username) {
usernameField.sendKeys(username);
}

Cannot be used when we are limiting the scope of driver or using iteration in xpath.

POM: Requires more code and manual element initialization, but allows greater flexibility.

Page Factory: Simplifies element initialization with PageFactory.initElements(), making it quicker to implement.

<table>
<tr><td>@FindBys</td><td>This annotation locates the web element by using the AND condition on the search criteria

@FindBys({
@FindBy(class="custom-control-check-box"),
@FindBy(id="game-chk-box")
})

WebElement chkBox;</td></tr>
<tr><td>@FindAll</td><td>Contrary to @FindBys, it uses an OR conditional relationship between the multiple @FindBy

@FindAll({
@FindBy(id="btn", //doesn't match
@FindBy(name="sbmtBtn"), //Matches
@FindBy(class="btn-primary") //doesn't match
})</td></tr>
<tr><td>@CacheLookUp</td><td>using @CacheLookUp, we can store the web elements in cache memory right after reading for the first time. It fastens our execution and the code, need not look up for the element on the web page and directly references it from memory.

@CacheLookUp
@FindBys({
@FindBy(class="custom-control-check-box"),
@FindBy(id="game-chk-box")
})

WebElement chkBox;</td></tr>
<tr><td>lazy load concept using the AjaxElementLocatorFactory</td><td>It can be used when your application uses Ajax elements. Additionally, you can use it while trying to find an element to perform an operation and pass a timeout value, until which the driver would wait before throwing an exception. In other words, it is a variant of implicit wait using the class AjaxElementLocatorFactory.

PageFactory.initElements(new AjaxElementLocatorFactory(driver, 20), this);</td></tr>
</table>
</td></tr>
<tr><td>Design Patterns</td><td>a. Page Object Model (POM)

<table>
<tr><td>Purpose</td><td>promotes a clean separation between the test code and the UI structure of the application.</td></tr>
<tr><td>How it works</td><td>Each page of the application is represented as a class, and all actions (methods) and elements (web elements) on the page are encapsulated within this class.</td></tr>
</table>

b. Factory Pattern(PageObjectManager)

<table>
<tr><td>Purpose</td><td>To create objects without exposing the instantiation logic, allowing dynamic object creation at runtime</td></tr>
<tr><td>Use Case in Selenium</td><td>A factory can be used to create WebDriver objects based on different browsers (Chrome, Firefox, etc.).</td></tr>
<tr><td>Code</td><td>public class WebDriverFactory {
public static WebDriver getDriver(String browserType) {</td></tr>
</table>
</td></tr>
</table>

```
        if (browserType.equalsIgnoreCase("chrome")) {
            return new ChromeDriver();
        } else if (browserType.equalsIgnoreCase("firefox")) {
            return new FirefoxDriver();
        }
        return null;
    }
}
```

Database

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class DatabaseTest {
    public static void main(String[] args) {
        // Database connection details
        String url = "jdbc:mysql://localhost:3306/yourDatabase";
        String user = "yourUsername";
        String password = "yourPassword";

        // Connect to the database
        try (Connection conn = DriverManager.getConnection(url, user, password)) {
            // Create a statement
            Statement stmt = conn.createStatement();

            // Execute a query
            ResultSet rs = stmt.executeQuery("SELECT * FROM yourTable");

            // Process the results
            while (rs.next()) {
                System.out.println(rs.getString("column1") + " " + rs.getString("column2"));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

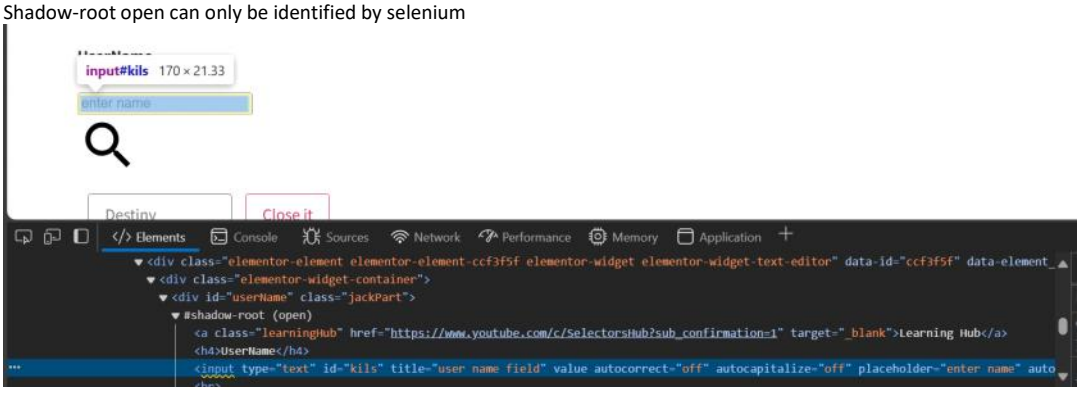
Shadow Dom

Components:

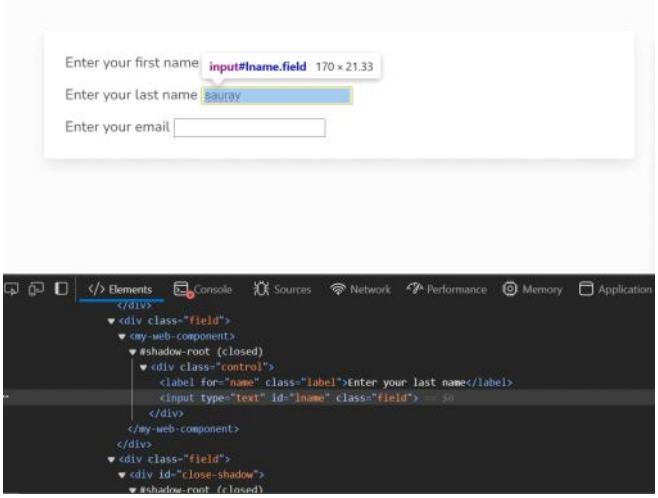
- Shadow Host: This is the regular DOM element to which a shadow DOM is attached.
- Shadow Tree: The DOM tree inside the shadow DOM. It is the root for the encapsulated DOM and CSS.
- Shadow Boundary: The boundary between the shadow DOM and the light DOM (the regular DOM outside the shadow DOM).
- Shadow Root: The root node of the shadow tree. It can be thought of as the entry point to the shadow DOM.

```
WebElement shadowHost = driver.findElement(By.id("userName"));
SearchContext shadowRoot = shadowHost.getShadowRoot();
WebElement usernameField = shadowRoot.findElement(By.cssSelector("#kils"));
usernameField.sendKeys("your_username");

//document.querySelector("#userName").shadowRoot.querySelector("#app2").shadowRoot.querySelector("#pizza")
WebElement shadowHost2 = shadowRoot.findElement(By.cssSelector("#app2"));
SearchContext shadowRoot2 = shadowHost2.getShadowRoot();
WebElement pizzaText = shadowRoot2.findElement(By.cssSelector("#pizza"));
pizzaText.sendKeys("Dominos");
```

Shadow-root open can only be identified by selenium



Closed Shadow Dom

	 js.executeScript("document.querySelector(\"my-web-component\").myRoot.querySelector('#lname').value=\"saurav\""); Quit Vs Close Close: Closes Active Window, session id will be maintained Quit: Closes all window, session id=null
What are some methods provided by the WebDriver interface?	Some common methods include get, quit, close, getWindowHandle, getTitle, etc.

scroll down to a specific element	JavascriptExecutor js = (JavascriptExecutor) driver;
	js.executeScript("arguments[0].scrollIntoView(true);", element);
	
	Actions actions = new Actions(driver);
	actions.moveToElement(element).build().perform();

How can you input text into a field without using sendKeys()	JavascriptExecutor js = (JavascriptExecutor) driver; js.executeScript("document.getElementById('elementId').value='Your Text';");
Get vs navigate	get(): Directly loads a new URL into the browser. navigate(): Provides more control over navigation, including: forward(): Navigates forward in the browser history. back(): Navigates backward in the browser history. refresh(): Reloads the current page. driver.get() : It's used to go to the particular website , But it doesn't maintain the browser History and cookies so , we can't use forward and backward button , if we click on that , page will not get schedule driver.navigate() : it's used to go to the particular website , but it maintains the browser history and cookies, so we can use forward and backward button to navigate between the pages during the coding of Testcase
Tooltips in Selenium	String tooltipText = element.getAttribute("title");
Default Timeout	Implicit Wait: 0 seconds (fails immediately if element not found) Page Load Timeout: 300 seconds (5 minutes) Async Script Timeout: 30 seconds
SHIFT+ENTER	String press = Keys.chord(Keys.SHIFT,Keys.ENTER); webelement.sendKeys(press);
getCssValue	String color = element.getCssValue("color"); Assert.assertEquals(color, "rgba(0, 0, 0, 1)");

Streams	List<String> originalList = elementsList.stream().map(s -> s.getText()).collect(Collectors.toList());
	List<String> sortedList = originalList.stream().sorted().collect(Collectors.toList());
	Assert.assertTrue(originalList.equals(sortedList));
	
	do {
	List<WebElement> rows = driver.findElements(By.xpath("//tr/td[1]"));
	price = rows.stream().filter(s -> s.getText().contains("Rice")).map(s -> getPriceVeggie(s))
	.collect(Collectors.toList());
	if (price.size() < 1) {
	driver.findElement(By.cssSelector("[aria-label='Next']")).click();
	}
	} while (price.size() < 1);

Browser Instantiation	We cannot instantiate the WebDriver interface directly as it is not a class. Browser-Specific Driver Instances: Example: FirefoxDriver driver = new FirefoxDriver(); This creates an object specific to the Firefox browser and allows invocation of only methods supported by FirefoxDriver. Scripts using this setup will run only on Firefox. For other browsers, individual driver objects must be created, e.g., ChromeDriver driver = new ChromeDriver(); or InternetExplorerDriver driver = new InternetExplorerDriver();.

Need for Cross-Browser Testing:
Using browser-specific objects limits flexibility. Cross-browser compatibility requires dynamic object referencing.

Dynamic Polymorphism:
Using WebDriver driver = new FirefoxDriver(); allows flexibility. Here, driver is a reference of WebDriver, and the object is FirefoxDriver. This setup enables calling methods defined in the WebDriver interface, regardless of the specific browser driver being used.
Other browser drivers like ChromeDriver, InternetExplorerDriver, etc., also implement the WebDriver interface via RemoteWebDriver.

Important Note:
Browser-specific methods cannot be invoked across different drivers. For example, a method specific to ChromeDriver cannot be called on a FirefoxDriver instance. Doing so will lead to script failure.

Options

In Selenium 3, capabilities were defined in a session by using Desired Capabilities classes. As of Selenium 4, you must use the browser options classes. For remote driver sessions, a browser options instance is required as it determines which browser will be used.

browserName	Browser name is set by default when using an Options class instance. ChromeOptions chromeOptions = new ChromeOptions(); String name = chromeOptions.getBrowserName();
browserVersion	This capability is optional, this is used to set the available browser version at remote end. In recent versions of Selenium, if the version is not found on the system, it will be automatically downloaded by Selenium Manager ChromeOptions chromeOptions = new ChromeOptions(); String version = "latest"; chromeOptions.setBrowserVersion(version);
pageLoadStrategy	The document.readyState property of a document describes the loading state of the current document. Strategy | Ready State | Purpose normal | complete | Used by default, waits for all resources to download eager | interactive | DOM access is ready, but other resources like images may still be loading none | Any | Does not block WebDriver at all When navigating to a new page via URL, by default, WebDriver will hold off on completing a navigation method (e.g., driver.navigate().get()) until the document ready state is complete. This does not necessarily mean that the page has finished loading, especially for sites like Single Page Applications that use JavaScript to dynamically load content after the Ready State returns complete. Note also that this behavior does not apply to navigation that is a result of clicking an element or submitting a form. If a page takes a long time to load as a result of downloading assets (e.g., images, css, js) that aren't important to the automation, you can change from the default parameter of normal to eager or none to speed up the session. This value applies to the entire session, so make sure that your waiting strategy is sufficient to minimize flakiness.
platformName	This identifies the operating system at the remote-end, fetching the platformName returns the OS name. In cloud-based providers, setting platformName sets the OS at the remote-end. ChromeOptions chromeOptions = new ChromeOptions(); String platform = "OS X 10.6"; chromeOptions.setPlatformName(platform);
acceptInsecureCerts	This capability checks whether an expired (or) invalid TLS Certificate is used while navigating during a session. If the capability is set to false, an insecure certificate error will be returned as navigation encounters any domain certificate problems. If set to true, invalid certificate will be trusted by the browser. All self-signed certificates will be trusted by this capability by default. Once set, acceptInsecureCerts capability will have an effect for the entire session. ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.setAcceptInsecureCerts(true);
Timeouts	A WebDriver session is imposed with a certain session timeout interval, during which the user can control the behaviour of executing scripts or retrieving information from the browser. Each session timeout is configured with combination of different timeouts as described below: Script Timeout | Specifies when to interrupt an executing script in a current browsing context. The default timeout 30,000 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS);

		chromeOptions.setScriptTimeout(duration);
	Page Load Timeout	Specifies the time interval in which web page needs to be loaded in a current browsing context. The default timeout 300,000 is imposed when a new session is created by WebDriver. If page load limits a given/default time frame, the script will be stopped by TimeoutException. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setPageLoadTimeout(duration);
	Implicit Wait Timeout	This specifies the time to wait for the implicit element location strategy when locating elements. The default timeout 0 is imposed when a new session is created by WebDriver. ChromeOptions chromeOptions = new ChromeOptions(); Duration duration = Duration.of(5, ChronoUnit.SECONDS); chromeOptions.setImplicitWaitTimeout(duration);
unhandledPromptBehavior	Specifies the state of current session's user prompt handler. Defaults to dismiss and notify state • dismiss • accept • dismiss and notify • accept and notify • ignore ChromeOptions chromeOptions = new ChromeOptions(); chromeOptions.setUnhandledPromptBehaviour(UnexpectedAlertBehaviour.DISMISS_AND_NOTIFY);	

proxy	Proxy proxy = new Proxy();
	proxy.setHttpProxy("<HOST:PORT>");
	ChromeOptions options = new ChromeOptions();
	options.setCapability("proxy", proxy);	

chromeOptions	Load Extention	ChromeOptions options = new ChromeOptions();
		Path path = Paths.get("src/test/resources/extensions/webextensions-selenium-example.crx");
		File extensionFilePath = new File(path.toUri());
		
		options.addExtensions(extensionFilePath);
		
		driver = new ChromeDriver(options);
		driver.get("https://www.selenium.dev/selenium/web/blank.html");
		WebElement injected = driver.findElement(By.id("webextensions-selenium-example"));
		Assertions.assertEquals(
		"Content injected by webextensions-selenium-example", injected.getText());

	Common Args	options.addArguments("--start-maximized");
		
		Commonly used args include --start-maximized, --headless=new and --user-data-dir=…
		options.addArguments("--blink-settings=imagesEnabled=false");

Do	• Selenium should not be used to prepare a test case. All repetitive actions and preparations for a test case, should be done through other methods. For example, most web UIs have authentication (e.g. a login form). Eliminating logging in via web browser before every test will improve both the speed and stability of the test. • Mock external services • Avoid sharing state: Do not share test data and Create a new WebDriver instance per test
Don't	• Captchas • File downloads • HTTP response codes • Gmail, email and Facebook logins • Test dependency • Performance testing • Link spidering • Two Factor Authentication
Test independency	Write each test as its own unit. Write the tests in a way that will not be reliant on other tests to complete

using a fluent API	public class GoogleSearchPage extends BasePage {
	public GoogleSearchPage(WebDriver driver) {
	super(driver);
	// Generally do not assert within pages or components.
	// Effectively throws an exception if the lambda condition is not met.
	new WebDriverWait(driver, Duration.ofSeconds(3)).until(d -> d.findElement(By.id("logo")));
	}
	
	public GoogleSearchPage setSearchString(String sstr) {
	driver.findElement(By.id("gbqfq")).sendKeys(sstr);
	return this;
	}

```
    public void clickSearchButton() {
        driver.findElement(By.id("gbqfb")).click();
    }
}
```

POM

The public methods represent the services that the page offers
Try not to expose the internals of the page
Generally don't make assertions
Methods return other PageObjects
Need not represent an entire page
Different results for the same action are modelled as different methods

```
public class LoginPage {
    private final WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;

        // Check that we're on the right page.
        if (!"Login".equals(driver.getTitle())) {
            // Alternatively, we could navigate to the login page, perhaps logging out first
            throw new IllegalStateException("This is not the login page");
        }
    }

    // The login page contains several HTML elements that will be represented as WebElements.
    // The locators for these elements should only be defined once.
        By usernameLocator = By.id("username");
        By passwordLocator = By.id("passwd");
        By loginButtonLocator = By.id("login");

    // The login page allows the user to type their username into the username field
    public LoginPage typeUsername(String username) {
        // This is the only place that "knows" how to enter a username
        driver.findElement(usernameLocator).sendKeys(username);

        // Return the current page object as this action doesn't navigate to a page represented by another PageObject
        return this;
    }

    // The login page allows the user to type their password into the password field
    public LoginPage typePassword(String password) {
        // This is the only place that "knows" how to enter a password
        driver.findElement(passwordLocator).sendKeys(password);

        // Return the current page object as this action doesn't navigate to a page represented by another PageObject
        return this;
    }

    // The login page allows the user to submit the login form
    public HomePage submitLogin() {
        // This is the only place that submits the login form and expects the destination to be the home page.
        // A seperate method should be created for the instance of clicking login whilst expecting a login failure.
        driver.findElement(loginButtonLocator).submit();

        // Return a new page object representing the destination. Should the login page ever
        // go somewhere else (for example, a legal disclaimer) then changing the method signature
        // for this method will mean that all tests that rely on this behaviour won't compile.
        return new HomePage(driver);
    }

    // The login page allows the user to submit the login form knowing that an invalid username and / or password were entered
    public LoginPage submitLoginExpectingFailure() {
        // This is the only place that submits the login form and expects the destination to be the login page due to login failure.
        driver.findElement(loginButtonLocator).submit();

        // Return a new page object representing the destination. Should the user ever be navigated to the home page after submiting a
login with credentials
        // expected to fail login, the script will fail when it attempts to instantiate the LoginPage PageObject.
        return new LoginPage(driver);
    }

    // Conceptually, the login page offers the user the service of being able to "log into"
    // the application using a user name and password.
    public HomePage loginAs(String username, String password) {
        // The PageObject methods that enter username, password & submit login have already defined and should not be repeated
here.
        typeUsername(username);
```

	``
	        typePassword(password);
	        return submitLogin();
	    }
	}
	``

Read PDf	```
	import org.apache.pdfbox.pdmodel.PDDocument;
	import org.apache.pdfbox.text.PDFTextStripper;
	import org.testng.annotations.Test;
	import java.io.File;
	import java.io.IOException;
	import java.util.List;
	
	public class PDFTest {
	
	    private static List<File> getListOfFiles(String directoryName) {
	        File directory = new File(directoryName);
	        File[] fList = directory.listFiles();
	        assert fList != null;
	        return List.of(fList);
	    }
	
	    @Test
	    public void pdfTest() {
	        String downloadPath = System.getProperty("user.home") + "/Downloads/";
	        List<File> files = getListOfFiles(downloadPath);
	        File downloadedFile = null;
	        for (File file : files) {
	            if (file.getName().endsWith(".pdf")) {
	                downloadedFile = file;
	                break;
	            }
	        }
	        if (downloadedFile != null) {
	            try {
	                PDDocument document = PDDocument.load(downloadedFile);
	                PDFTextStripper stripper = new PDFTextStripper();
	                String pdfText = stripper.getText(document);
	                if (pdfText.contains("EPAM")) {
	                    System.out.println("PDF downloaded and validation passed.");
	                } else {
	                    System.out.println("PDF downloaded but validation failed. Expected text not found.");
	                }
	                document.close();
	            } catch (IOException e) {
	                System.err.println("Error reading PDF: " + e.getMessage());
	            }
	            if (downloadedFile.delete()) {
	                System.out.println("Downloaded file deleted.");
	            } else {
	                System.err.println("Failed to delete downloaded file.");
	            }
	        } else {
	            System.out.println("Downloaded PDF not found.");
	        }
	    }
	}
	``

Autowiring PageObjectManager	You can automate the creation of boilerplate code for your Page Object Manager using reflection and annotations, similar to how Spring Boot handles dependency injection. Here's how you can do this: ### Steps: 1. **Define a Marker Annotation for Pages**: Create a custom annotation to mark your page classes. 2. **Scan for Annotated Classes**: Use reflection to find all classes annotated with this marker. 3. **Instantiate Pages Dynamically**: Dynamically instantiate and cache these classes as needed. Here’s an updated approach to your `PageObjectManager`: --- ### 1. Create a Marker Annotation ```java package annotations; import java.lang.annotation.ElementType; import java.lang.annotation.Retention; import java.lang.annotation.RetentionPolicy; import java.lang.annotation.Target; @Target(ElementType.TYPE)

```
@Retention(RetentionPolicy.RUNTIME)
public @interface Page {
}
```

---

2. Annotate Your Page Classes

```java
package pages;

import annotations.Page;
import org.openqa.selenium.WebDriver;

@Page
public class HomePage {
    private WebDriver driver;

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }
}
```

Repeat this for other page classes, e.g., `LoginSignupPage`, `SignUpPage`, etc.

---

3. Update `PageObjectManager`

```java
package pages;

import annotations.Page;
import org.openqa.selenium.WebDriver;

import java.util.HashMap;
import java.util.Map;

public class PageObjectManager {
    private final WebDriver driver;
    private final Map<Class<?>, Object> pageCache = new HashMap<>();

    public PageObjectManager(WebDriver driver) {
        this.driver = driver;
    }

    @SuppressWarnings("unchecked")
    public <T> T getPage(Class<T> pageClass) {
        return (T) pageCache.computeIfAbsent(pageClass, clazz -> {
            if (!clazz.isAnnotationPresent(Page.class)) {
                throw new IllegalArgumentException("Class " + clazz.getName() + " is not annotated with @Page");
            }
            try {
                return clazz.getDeclaredConstructor(WebDriver.class).newInstance(driver);
            } catch (Exception e) {
                throw new RuntimeException("Failed to create instance of " + clazz.getName(), e);
            }
        });
    }
}
```

---

4. Use the `PageObjectManager`

```java
PageObjectManager manager = new PageObjectManager(driver);

// Access pages dynamically
HomePage homePage = manager.getPage(HomePage.class);
LoginSignupPage loginSignupPage = manager.getPage(LoginSignupPage.class);
```

Benefits:
1. **Dynamic Instantiation**: No need to manually define getter methods for each page.
2. **Scalable**: Adding a new page only requires annotating it with `@Page`.
3. **Cleaner Code**: Reduces boilerplate and centralizes instantiation logic.
4. **Easy to Extend**: You can add additional features like lazy initialization or logging.

This design mimics Spring Boot's autowiring feature, providing a clean, automated approach to manage your page objects.

WebDriverListener

The WebDriverListener interface in Selenium is a valuable tool for enhancing the visibility and debugging capabilities of your test automation scripts. It allows you to intercept and log various events that occur during WebDriver interactions, providing insights into the execution flow and potential issues.

Key Benefits:

- Enhanced Debugging: By logging events like navigation, element clicks, and waits, you can easily identify the root cause of t est failures.
- Improved Test Reporting: You can customize reports to include detailed information about test execution, including screenshots, logs, and performance metrics.
- Increased Test Reliability: By monitoring events like page loads and script executions, you can proactively identify and addr ess potential performance bottlenecks or errors.

```
public class MyListener implements WebDriverListener {
  private final WebDriver driver;

  public MyListener(WebDriver driver) {
    this.driver = driver;
  }

  @Override
  public void beforeGetTitle(WebDriver driver) {
    System.out.println("Getting the title");
  }

  @Override
  public void beforeClick(WebElement element) {
    System.out.println(element + " is going to be clicked");
    // takeScreenshot("before_click");
  }

  @Override
  public void afterClick(WebElement element) {
    System.out.println(element + " was clicked");
    takeScreenshot("after_click");
  }

  @Override
  public void onError(Object target, Method method, Object[] args, InvocationTargetException e) {
    takeScreenshot("On Error");
  }

  private void takeScreenshot(String phase) {
    File screenshot = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
    try {
      String fileName = "screenshot_" + phase + "_" + System.currentTimeMillis() + ".png";
      FileUtils.copyFile(screenshot, new File(fileName));
      System.out.println("Screenshot captured: " + fileName);
    } catch (IOException e) {
      System.err.println("Failed to save screenshot: " + e.getMessage());
    }
  }
}


  @Test
  public void webdriverListenerTest(){
    WebDriver driver=new EdgeDriver();
    WebDriverListener myListner = new MyListener(driver);
    driver=new EventFiringDecorator<>(myListner).decorate(driver);
    driver.get("https://www.google.com/");
    System.out.println("Page Loaded");
    System.out.println("Title is "+driver.getTitle());
    try {
      driver.findElement(By.xpath("/html/body/div[1]/div[3]/form/div[1]/div[1]/div/sectionofthedevil")).click();
    }catch (Exception e){
      //e.printStackTrace();
    }
    driver.close();
  }
}
```

Log4j2.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<Configuration status="INFO">
  <Properties>
    <Property name="basePath">./logs</Property>
  </Properties>
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
    </Console>
    <RollingFile name="File" fileName="${basePath}/automation.log"
            filePattern="${basePath}/automation-%d{yyyy-MM-dd}.log">
      <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
      <SizeBasedTriggeringPolicy size="50000"/>
    </RollingFile>
  </Appenders>
  <Loggers>
    <Root level="Info"><!-- Here you can use multiple log levels All < Trace < Debug < Info < Warn < Error < Fatal < Off -->
      <AppenderRef ref="File"/>  <!-- <AppenderRef ref="Console"/> -->
    </Root>
  </Loggers>
</Configuration>
```

<Configuration status="WARN">	• The <Configuration> tag is the root element that encapsulates all Log4j2 configurations.
	• The status attribute controls Log4j2's internal debugging logs, which are useful for diagnosing configuration issues.
	• Options for status:
	  • OFF: No internal logs from Log4j2.
	  • ERROR: Logs only errors in Log4j2 configuration.
	  • WARN (default): Logs warnings and errors about configuration issues.
	  • INFO: Provides more detailed information, such as component initialization.
	  • DEBUG: Outputs detailed logs, including internal processing of Log4j2.
	  • TRACE: Most verbose, showing fine-grained details of Log4j2 operations

<Properties> <Property name="basePath">./logs</Property> </Properties>	Defines reusable key-value pairs for configuration. These properties can be referenced elsewhere in the file using ${propertyName}. Common Usage: Set file paths, environment variables, or dynamic configurations. Example: ${env:USER_HOME} to get the user's home directory.

<Appenders>
  ...
</Appenders>
	<table><tr><td>Console Appender</td><td><Console name="Console" target="SYSTEM_OUT"> <PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/>
	</Console>
	
	Purpose: Logs messages to the console (stdout or stderr).
	Options for target:
	SYSTEM_OUT: Writes logs to standard output (stdout).
	SYSTEM_ERR: Writes logs to standard error (stderr).</td></tr><tr><td>File Appender</td><td><RollingFile name="File" fileName="${basePath}/automation.log"
	  filePattern="${basePath}/automation-%d{yyyy-MM-dd}.log">
	  ...
	</RollingFile>
	
	Purpose: Writes logs to files and supports file rollover.
	
	Attributes:
	fileName: The current log file.
	filePattern: The pattern for rollover files (e.g., ${basePath}/automation-%d{yyyy-MM-dd}.log for daily logs).
	
	Triggering Policies:
	  • SizeBasedTriggeringPolicy: Rolls over when the file exceeds a specific size:<SizeBasedTriggeringPolicy size="50MB"/>
	  • TimeBasedTriggeringPolicy: Rolls over at specific time intervals.:<TimeBasedTriggeringPolicy interval="1" modulate="true"/>
	  • CompositeTriggeringPolicy:
	<CompositeTriggeringPolicy>
	  <Policies>
	  <SizeBasedTriggeringPolicy size="50MB"/>
	  <TimeBasedTriggeringPolicy interval="1"/>
	  </Policies>
	</CompositeTriggeringPolicy></td></tr></table>

PatternLayout	<PatternLayout pattern="%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n"/> Common Pattern Elements: %d{pattern}: Date and time (e.g., %d{yyyy-MM-dd HH:mm:ss}). %t: Thread name.

<table>
<tr><td></td><td>
<table>
<tr><td></td><td>%level: Log level (INFO, DEBUG, etc.).
%logger{length}: Logger name truncated to the specified length.
%msg: The actual log message.
%n: Newline.</td></tr>
<tr><td>Loggers Section</td><td>
<table>
<tr><td>Root</td><td><Root level="Info">
  <AppenderRef ref="File"/>
</Root>

The root logger applies globally to all loggers unless overridden.
Attributes:
level: Minimum level of logs to capture (e.g., DEBUG, INFO, WARN).
AppenderRef: Specifies which appenders the logger uses.</td></tr>
<tr><td>Custom Loggers</td><td><Logger name="com.example" level="DEBUG" additivity="false">
  <AppenderRef ref="Console"/>
</Logger>

Attributes:
name: Logger name (usually a package or class name).
level: Minimum log level for this logger.
additivity: Whether to inherit appenders from parent loggers.</td></tr>
</table>

Change Log Levels:

Update the level attribute in the <Root> or <Logger> tags.
Levels:
ALL: Log everything.
TRACE: Very fine-grained logs for debugging.
DEBUG: Logs used for debugging purposes.
INFO: Informational messages.
WARN: Warnings that might lead to issues.
ERROR: Error messages.
FATAL: Critical errors that might cause the application to stop.
OFF: Disables logging.
Add More Appenders:

Log to different destinations like databases, emails, or remote servers.
Use Asynchronous Logging:

Add <Async> appender for performance improvements.
Dynamic Properties:

Use ${env:} or ${sys:} to dynamically set properties.
Categorized Logging:

Define custom <Logger> tags for specific classes or packages.</td></tr>
</table>
</td></tr>
<tr><td>disable images in selenium</td><td>Map<String, Object> prefs = new HashMap<>();
  prefs.put("profile.managed_default_content_settings.images", 2);
  options.setExperimentalOption("prefs", prefs);</td></tr>
<tr><td>Extent Reports</td><td>public class ExtentReportManager {

  public static ExtentReports extentReports;

  public static ExtentReports createInstance(String fileName, String reportName, String documentTitle) {
  ExtentSparkReporter extentSparkReporter = new ExtentSparkReporter(fileName);
  extentSparkReporter.config().setReportName(reportName);
  extentSparkReporter.config().setDocumentTitle(documentTitle);
  extentSparkReporter.config().setTheme(Theme.STANDARD);
  extentSparkReporter.config().setEncoding("utf-8");
  extentReports = new ExtentReports();
  extentReports.attachReporter(extentSparkReporter);
  return extentReports;
  }

  public static String getReportNameWithTimeStamp() {
  DateTimeFormatter dateTimeFormatter = DateTimeFormatter.ofPattern("yyyy_MM_dd_HH_mm_ss");
  LocalDateTime localDateTime = LocalDateTime.now();
  String formattedTime = dateTimeFormatter.format(localDateTime);
  return "TestReport" + formattedTime + ".html";
  }

  public static void logPassDetails(String log) {
  ExtentReportConfigListener.extentTest.get().pass(MarkupHelper.createLabel(log, ExtentColor.GREEN));
  }
  public static void logFailureDetails(String log) {
  ExtentReportConfigListener.extentTest.get().fail(MarkupHelper.createLabel(log, ExtentColor.RED));</td></tr>
</table>

```
}}



public class ExtentReportConfigListener implements ITestListener {

	private static ExtentReports extentReports;
	public static ThreadLocal<ExtentTest> extentTest = new ThreadLocal<>();

	public void onStart(ITestContext context) {
		String fileName = ExtentReportManager.getReportNameWithTimeStamp();
		String fullReportPath = System.getProperty("user.dir") + "\\reports\\" + fileName;
		extentReports = ExtentReportManager.createInstance(fullReportPath, "Test API Automation Report",
			"Test ExecutionReport");
	}

	public void onFinish(ITestContext context) {
		if (extentReports != null)
			extentReports.flush();
	}

	public void onTestStart(ITestResult result) {
		ExtentTest test = extentReports.createTest(
			"Test Name " + result.getTestClass().getName() + " - " + result.getMethod().getMethodName());
		extentTest.set(test);
	}

	public void onTestFailure(ITestResult result) {
		ExtentReportManager.logFailureDetails(result.getThrowable().getMessage());
		String stackTrace = Arrays.toString(result.getThrowable().getStackTrace());
		stackTrace = stackTrace.replaceAll(",", "<br>");
		String formmatedTrace = "<details>\n" + "    <summary>Click Here To See Exception Logs</summary>\n" + "    "
			+ stackTrace + "\n" + "</details>\n";
		ExtentReportManager.logExceptionDetails(formmatedTrace);
	}
}
```

Cucumber

02 March 2024 07:45

Syntax	@SmokeTest @RegressionTest Scenario: Successful Login Given: Preconditions are mentioned in the Given keyword. When: The purpose of the When Steps is to describe the user action. Then: The purpose of Then Steps is to observe the expected output. The observations should be related to the business value/benefit of your Feature description. And: Used to add additional conditions or actions to Given, When, or Then steps. But: Used to add conditions or actions that are exceptions to the previous steps.
Testng Runner	AbstractTestNGCucumberTests needs to be inherited by the runner
Tagging	A feature or scenario can have as many tags as you like. Separate them with spaces Tags can be placed above the following Gherkin elements: • Feature • Scenario • Scenario Outline • Examples Scenario Outline: Steps will run conditionally if tagged Given user is logged in When user clicks <link> Then user will be logged out @mobile Examples: | link | | logout link on mobile | @desktop Examples: | link | | logout link on desktop | It is not possible to place tags above Background or steps (Given, When, Then, And and But). Tags are inherited by child elements. Tags that are placed above a Feature will be inherited by Scenario, Scenario Outline, or Examples. Tags that are placed above a Scenario Outline will be inherited by Examples.
And Tag in Test Runner	cucumber --tags "@smoke and @regression"
Or Tag in Test Runner	cucumber --tags "@smoke or @regression"
NotTag in Test Runner	cucumber --tags "not @wip" (@smoke or @ui) and (not @slow)
Hooks	<table><tr><td>Scenario hooks</td><td>Scenario hooks run for every scenario. • Before hooks run before the first step of each scenario • After hooks run after the last step of each scenario, even when the step result is failed, undefined, pending, or skipped</td></tr><tr><td>Step hooks</td><td>Step hooks are invoked before and after a step. The hooks have ‘invoke around’ semantics, meaning that if a BeforeStep hook is executed the AfterStep hooks will also be executed regardless of the result of the step. If a step did not pass, the following step and its hooks will be skipped • BeforeStep • AfterStep</td></tr><tr><td>Conditional hooks</td><td>Hooks can be conditionally selected for execution based on the tags of the scenario. To run a particular hook only for certain scenarios, you can associate a Before or After hook with a tag expression. @After("@browser and not @headless") public void doSomethingAfter(Scenario scenario){ }</td></tr><tr><td>Global hooks</td><td>Global hooks will run once before any scenario is run or after all scenario have been run. BeforeAll run before any scenario is run. AfterAll run after all scenarios have been executed.</td></tr></table>
Background	It runs before each and every scenario were for a feature in which it is defined
Hooks Vs Background	Hook runs before Background Before Hooks->Background->Scenario->After Hook

	Background cannot be tagged, hooks can be: @Before("@Smoke") Orders Hooks: @Before(order=0),@Before(order=1)

{string} Regex in feature File	{string} matches quoted strings: "text"
	
	Given the user is on the "login" page
	
	@Given("the user is on the {string} page")
	public void theUserIsOnThePage(String page) {
	System.out.println("User is on the " + page + " page");
	}

(.*) Regex in feature File	(.*) matches any character sequence, including spaces, without needing quotes
	
	Given the user is on the "login" page
	
	@Given("^the user is on the (.*) page$")
	public void theUserIsOnThePage(String page) {
	System.out.println("User is on the " + page + " page");
	}

{int}	Given the user has "42" points
{float}	Given the price is "3.14"
{word}	Matches a single word (a sequence of characters without spaces). Given the user is logged in as username

Scenario Outline	When The "AddComment" request is sent with the "Post" HTTP method on IssueID as "<ISSUEID>"
	
	Examples:
	| RowNumber | ISSUEID | DISPLAY_NAME | UPDATE_AUTHOR_EMAIL_ADDRESS | COMMENT |
	| 1 | ISSUEID | DISPLAY_NAME | UPDATE_AUTHOR_EMAIL_ADDRESS | COMMENT |
	| 2 | ISSUEID | DISPLAY_NAME | UPDATE_AUTHOR_EMAIL_ADDRESS | COMMENT |
	
	@When("The {string} request is sent with the {string} HTTP method on IssueID as {string}")
	public void request_is_sent_with_http_method_on_issueid(String resource, String httpMethod, String issueID) {
	issueID = excelSheetReader.readCell(issueID);
	if (httpMethod.equalsIgnoreCase("POST")) {
	addCommentResponse = SpecBuilders.SendPostAndReturnResponse(updatedJsonPayload, issueID, resource);
	}
	}

Dependency Injection	Dependency Injection (DI) in Cucumber using PicoContainer allows you to share state and objects between step definitions, hooks, and other classes. • **No Manual Management**: Dependency management is handled by PicoContainer, so you don't need to manually manage object lifecycles or dependencies between steps. • **Consistency Across Steps**: Objects maintain their state across different steps within the same scenario, making tests easier to write and maintain. • **Simple Integration**: Integrating PicoContainer with Cucumber requires minimal setup and aligns with Cucumber's aim to simplify writing BDD style tests. Using PicoContainer to share state between steps in a scenario is easy and non-intrusive. All you need is a constructor that requires an object that PicoContainer can create and inject. PicoContainer is invisible. Add a dependency to cucumber-picocontainer and make sure that the constructors for the step classes require an instance of the same class. Simply hand it some classes and it will instantiate each one, correctly wired together via their constructors. That’s it. Cucumber scans your classes with step definitions in them, passes them to PicoContainer, then asks it to create new instances for every scenario. • Add PicoContainer to the Project • Create a Test Context class which will hold all the objects state • Divide the Steps class into multiple steps classes with logical separation • Write Constructor to share Test Context

Add Dependency	<dependencies> <dependency> <groupId>io.cucumber</groupId> <artifactId>cucumber-picocontainer</artifactId> <version>7.0.0</version> <scope>test</scope> </dependency> </dependencies>
Create Shared Class	Define a class that will hold the shared state or objects. public class SharedState { private String importantData;

		public void setImportantData(String data) {
		this.importantData = data;
		}
		
		public String getImportantData() {
		return importantData;
		}
		}

	Feature File	Feature: Shared State Example Scenario: Sharing state between steps Given I set the important data to "expectedData" in step definition 1 Then I should have access to the important data in step definition 2

	Step Definition with Dependency Injection	import io.cucumber.java.en.Given;
		
		public class StepDefinitions1 {
		private final SharedState sharedState;
		
		public StepDefinitions1(SharedState sharedState) {
		this.sharedState = sharedState;
		}
		
		@Given("I set the important data to {string} in step definition 1")
		public void setImportantData(String data) {
		sharedState.setImportantData(data);
		}
		}
		
		---------- 	
		
		import io.cucumber.java.en.Then;
		import static org.junit.Assert.assertEquals;
		
		public class StepDefinitions2 {
		private final SharedState sharedState;
		
		public StepDefinitions2(SharedState sharedState) {
		this.sharedState = sharedState;
		}
		
		@Then("I should have access to the important data in step definition 2")
		public void verifyImportantData() {
		String data = sharedState.getImportantData();
		System.out.println("The important data is: " + data);
		assertEquals("Expected data not found", "expectedData", data);
		}
		}

	Output	The important data is: expectedData

DI-Selenium Example	Create a Test Context class	public class TestContext {
		private WebDriverManager webDriverManager;
		private PageObjectManager pageObjectManager;
		
		public TestContext(){
		webDriverManager = new WebDriverManager();
		pageObjectManager = new PageObjectManager(webDriverManager.getDriver());
		}
		
		public WebDriverManager getWebDriverManager() {
		return webDriverManager;
		}
		
		public PageObjectManager getPageObjectManager() {
		return pageObjectManager;
		}
		
		}

	Divide the Steps file	OLD Step File public class HomePageSteps { WebDriver driver;

```
        HomePage homePage;
        PageObjectManager pageObjectManager;
        WebDriverManager webDriverManager;


        @Given("^user is on Home Page$")
        public void user_is_on_Home_Page(){
                webDriverManager = new WebDriverManager();
                driver = webDriverManager.getDriver();
                pageObjectManager = new PageObjectManager(driver);
                homePage = pageObjectManager.getHomePage();
                homePage.navigateTo_HomePage();
        }

        @When("^he search for \"([^\"]*)\"$")
        public void he_search_for(String product)  {
                homePage.perform_Search(product);
        }

}
```

Constructor for Step Definition classes to share TestContext

We need WebDriverManager & PageObjectManager in every step file, otherwise, you need to create objects for both classes using new operator again and again, which is Kill Bill.

Now with just adding Constructor to HomePageSteps file and pass TestContext as a Parameter to constructor would take all the pain. Within the TestContext object we have everything available which is required for the test. So now let's see how the new HomePageSteps class would look.

```
package stepDefinitions;

import cucumber.TestContext;
import cucumber.api.java.en.Given;
import cucumber.api.java.en.When;
import pageObjects.HomePage;

public class HomePageSteps {

        TestContext testContext;
        HomePage homePage;

        public HomePageSteps(TestContext context) {
                testContext = context;
                homePage = testContext.getPageObjectManager().getHomePage();
        }

        @Given("^user is on Home Page$")
        public void user_is_on_Home_Page(){
                homePage.navigateTo_HomePage();
        }

        @When("^he search for \"([^\"]*)\"$")
        public void he_search_for(String product)  {
                homePage.perform_Search(product);
        }
}
```

DataTable

```
List<Map<String, String>> rows = dataTable.asMaps(String.class, String.class);
    for (Map<String, String> row : rows) {
      String field = row.get("Field");
      String expectedValue = excelSheetReader.readCell(row.get("Value"));
      String actualValue = js.get(jsonpaths.getJsonPathForField(field));
      Assert.assertEquals(expectedValue, actualValue);
    }
```

Running Tests in Parallel

```
public class TestNGTestRunner extends AbstractTestNGCucumberTests {
  @Override
  @DataProvider(parallel = true)
  public Object[][] scenarios() {
    return super.scenarios();
  }
}
```

Add Surefire plugin. The default thread count of the dataprovider in parallel mode is 10. To change this the dataproviderthreadcount property needs to be added to the configuration section of the Surefire plugin in the POM.

```
 <build>
```

	<plugins> <plugin> <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-compiler-plugin</artifactId> <version>3.11.0</version> <configuration> <source>21</source> <target>21</target> <encoding>UTF-8</encoding> </configuration> </plugin> <plugin> <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-surefire-plugin</artifactId> <version>3.2.2</version> <configuration> <suiteXmlFiles> <suiteXmlFile>testng.xml</suiteXmlFile> </suiteXmlFiles> <properties> <property> <name>dataproviderthreadcount</name> <value>2</value> </property> </properties> </configuration> </plugin> </plugins> </build> This will run tests within same feature file in parallel. To run tests across feature file in paralle use testng.xml <?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Suite" parallel="classes" thread-count="2">> <test name="Test">> <classes> <class name="TestRunner.TestRunner"/> <class name="TestRunner.TestRunner2"/> </classes> </test> <!-- Test --> </suite> <!-- Suite -->

@skip	@Given("User is on the login page")
	@Skip
	public void userIsOnLoginPage() {
	// Step implementation
	}

run failed test cases in cucumber	@RunWith(Cucumber.class)
	@CucumberOptions(
	features = "src/test/resources/features",
	glue = "stepDefinitions",
	plugin = {"pretty", "html:target/cucumber-reports", "rerun:target/rerun.txt"}
	)
	public class RunCucumberTest {
	}
	
	New Runner
	
	@RunWith(Cucumber.class)
	@CucumberOptions(
	features = "@target/rerun.txt",
	glue = "stepDefinitions"
	)
	public class RerunFailedTests {
	}

Cucumber Options	Cucumber Options are used to configure properties for Cucumber tests

Options Type	Purpose	Default Value
dryRun	true: Checks if all the Steps have the Step Definition	false
features	set: The paths of the feature files	{}
glue	set: The paths of the step definition files	{}
tags	instruct: What tags in the features files should be executed	{}
monochrome	true: Display the console Output in much readable way	false
format	set: What all report formaters to use	false
strict	true: Will fail execution if there are undefined or pending steps	false

TestNG

17 February 2024 13:22

Test Annotation	1. @Test: Marks a method as a test method 2. @BeforeSuite: Marks a method to run before any test suite 3. @AfterSuite: Marks a method to run after all tests in a suite have run 4. @BeforeTest: Marks a method to run before any test method belonging to the classes inside the <test> tag in XML 5. @AfterTest: Marks a method to run after all test methods belonging to the classes inside the <test> tag in XML 6. @BeforeClass: Marks a method to run before the first test method in the current class is invoked 7. @AfterClass: Marks a method to run after all the test methods in the current class have been run 8. @BeforeMethod: Marks a method to run before each test method 9. @AfterMethod: Marks a method to run after each test method <suite> <test> <classes> <class> <methods>

@DataProvider	Marks a method as a data provider for test methods. It provides data to a test method.
	
	@DataProvider(name = "simpleData")
	public Object[][] createData() {
	return new Object[][] {
	{ "John", 25 },
	{ "Jane", 30 },
	{ "Alex", 22 }
	};
	}
	
	@Test(dataProvider = "simpleData")
	public void testWithDataProvider(String name, int age) {
	System.out.println("Name: " + name + ", Age: " + age);
	}
	
	DP in another file: @Test(dataProvider = "flights",dataProviderClass= DataProviders.class)

@Parameters	Marks a method parameter as a parameter for the test method. It allows passing parameters to test methods from the TestNG XML file.
	
	<suite name="Suite">
	<test
	name="Test">
	<parameter name="username" value="user123" />
	<parameter name="password" value="secret123" />
	<classes>
	<class name="com.example.MyTestClass" />
	</classes>
	</test> </suite>
	
	@Parameters({"username", "password"})
	@Test
	public void testLogin(String username, String password) {
	System.out.println("Username: " + username);
	System.out.println("Password: " + password);
	
	// Your test logic here using the provided parameters
	}

Disable a test	@Test(enabled=false)
Timeout	@Test(timeOut=10000) Or <?xml version="1.0" encoding="UTF-8"?> <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"> <suite name="Suite"> <test thread-count="5" name="Test" **time-out="5000"**> <classes> <class name="testRunner.TestNGRunner"/>

	``` </classes> </test> <!-- Test --> </suite> <!-- Suite --> ```
Exclude Tests	``` <test name="Test"> <classes> <class name="com.example.MyTestClass">** **<methods> **<exclude name="testMethodToExclude"/> **</methods> **</class> **</classes> **</test> We can also include Regex: <methods> <exclude name="test.*"/> </methods> ```

Exclude Tests Based on Groups	```
	@Test(groups = {"smoke"}) public void testMethod1() { // Test code for smoke group }
	
	<test name="Test">**
	**<groups>
	<run>
	**<exclude name="smoke"/>
	**<exclude name="regression"/>
	</run>
	**</groups>
	**<classes>
	**<class name="com.example.MyTestClass"/>
	**</classes>
	</test>
	``

Run Whole Package	``` <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd"> <suite name="Suite">** **<test name="Test">** **<packages> **<package name="com.example.tests"/> **</packages> **</test> </suite> ```

Test Tagging	```
	@Test(groups = {"smoke", "regression"})
	**public void testMethod1() {
	**System.out.println("Test Method 1");
	**}
	
	<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
	<suite name="Test Suite">**
	<test name="Test Methods">
	**<groups>
	**<run>
	**<include name="smoke"/>
	**</run>
	**</groups>
	**<classes>
	**<class name="org.saurav.TestSetup"/>
	**<class name="org.saurav.TestMethods"/>
	**<class name="org.saurav.TestExecution"/>
	**</classes>
	**</test>
	</suite>
	
	to run beforeclass/method/test when using groups give @BeforeClass(alwaysRun = true)
	
	**@BeforeClass(alwaysRun = true)
	**public void beforeClass() {
	**System.out.println("Before Class - Executed before the first test method in the current class.");
	**}
	
	**@AfterClass(alwaysRun = true)
	**public void afterClass() {
	**System.out.println("After Class - Executed after all test methods in the current class.");
	**}
	``

dependsOnMethods	@Test(groups = {"smoke"},dependsOnMethods = {"testMethod2"}) else <!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd" > <suite name="TestNG XML Dependency Suite" > <test name="ToolsQA" > <groups> <dependencies> <group depends-on= "openbrowser" name= "login"></group> </dependencies> </groups> <classes> <class name="GroupDependency" /> </classes> </test> </suite>
dependsOnGroups	@Test(groups ={ "groupB"}, dependsOnGroups = {"groupA", "group2"})
Order of Execution in Inheritance	TestNG runs @Before methods from the highest superclass to the subclass, and @After methods from the subclass back to the highest superclass
preserve-order	TestNG will run your tests in the order they are found in the XML file. If you want the classes and methods listed in this file to be run in an unpredictable order, set the preserve-order attribute to false <test name="Regression1" preserve-order="false"> <classes> <class name="test.Test1"> <methods> <include name="m1"/> <include name="m2"/> </methods> </class> <class name="test.Test2"/> </classes> </test>

@Ignore	@Ignore
	public class TestcaseSample {
	
	  @Test
	  public void testMethod1() {
	  }
	
	  @Test
	  public void testMethod2() {
	  }
	}

Rerunning failed tests	<table><tr><td>Using testng-failed.xml</td><td>• Run your TestNG suite: After the initial run, TestNG generates a testng-failed.xml file in the test-output folder. This file contains the list of failed test cases.
	• Execute testng-failed.xml: Right-click on this file and run it as a TestNG suite. This will re-execute only the failed test cases.</td></tr><tr><td>Using IRetryAnalyzer</td><td>• Create a RetryAnalyzer class: Implement the IRetryAnalyzer interface and define a retry() method. This method determines whether a test case should be re-tried based on a certain condition (e.g., maximum retry count).
	• Apply the RetryAnalyzer to Test Methods: Use the retryAnalyzer attribute in the @Test annotation to specify the RetryAnalyzer class.
	• Run your TestNG suite: When a test case fails, the RetryAnalyzer will attempt to re-run it up to the specified maximum number of retries.
	
	----Setup RetryAnalyzer
	import org.testng.IRetryAnalyzer;
	import org.testng.ITestResult;
	
	public class RetryAnalyzer implements IRetryAnalyzer {
	  private int count = 0;
	  private static final int maxRetryCount = 3;
	
	  public boolean retry(ITestResult result) {
	  if (count < maxRetryCount) {
	  count++;</td></tr></table>

```
            return true;
        }
        return false;
    }
}

----Applying RetryAnalyzer to a Test Method

@Test(retryAnalyzer = RetryAnalyzer.class)
public void testMethod() {
    // Test logic here
}
```

@Factory Annotation

```
import org.testng.annotations.Factory;

public class TestFactory {

    @Factory
    public Object[] factoryMethod() {
        return new Object[] { new TestClass(1), new TestClass(2) };
    }
}

// Test class
import org.testng.annotations.Test;

public class TestClass {
    private int parameter;

    public TestClass(int parameter) {
        this.parameter = parameter;
    }

    @Test
    public void testMethod() {
        System.out.println("Test method is running with parameter: " + parameter);
        // Add your test logic here using the 'parameter'
    }
}
```

@Test(invocationCount = 3)

```
InvocationCount is used when we want to run the same test multiple times.

 threadPoolSize controls how many threads TestNG will use to execute the
 invocations of a test method
 Ex: @Test(threadPoolSize = 3, invocationCount = 6)
 public void testMethod() {
 // Test logic
```

TestNG Priorities

```
@Test (priority = 1)

If we have methods with priority 0,1,2,3 and some without any priority,
First: 0 Priority
Second: No priority
Third: 1

In case of conflict, alphabetical order of testName
```

Soft Assert

```
SoftAssert softassert = new SoftAssert();
softassert.assertEquals(originalTitle, expectedTitle);
softassert.assertAll();
```

Parallel

```
    Methods: This will run the parallel tests on all @Test methods in TestNG.
    Tests: All the test cases present inside the <test> tag will run with this value.
    Classes: All the test cases present inside the classes that exist in the XML will run in parallel.
    Instances: This value will run all the test cases parallelly inside the same instance.

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Suite" parallel="methods" thread-count="2">
    <test name="Test">
        <parameter name="userName" value="saurav"/>
        <classes>
            <class name="TestRunner.TestNGTest"/>
        </classes>
```

	``` </test> <!-- Test --> <test name="Test2"> <parameter name="userName" value="singh"/> <classes> <class name="TestRunner.TestNGTest2"/> </classes> </test> </suite> <!-- Suite --> ```

exception handling	```
	  @Test(expectedExceptions = NumberFormatException.class)
	  public void testNumberFormatException() {
	  Integer.parseInt("abc"); // This will throw a NumberFormatException
	  }
	``
	This test will pass if exception occurs and fail when it doesn't

@Test (enabled = False)	
preserve-order in testng.xml	``` <!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd" > <suite name="preserve-order Suite"> <test name="preserve-order Test" preserve-order="true"> <classes> <class name="Testing_Pack.Test_One"/> <class name="Testing_Pack.Test_Two"/> <class name="Testing_Pack.Test_Three"/> </classes> </test> </suite> ``` Tests will run as One,Two,Three, else it would have run as One,three,two
thread count	In TestNG, the default thread count for parallel execution is 5

ITestContext	ITestContext is used to store and share data across the tests in selenium by using TestNG framework.
	
	This interface provides information about the current test execution context. It allows you to retrieve data such as the suite name, test name, test parameters, and test status.
	
	• getSuite(): Get information about the test suite.
	• getName(): Get the name of the test.
	• getStartDate(): Get the start time of the test execution.
	• getEndDate(): Get the end time of the test execution.
	• setAttribute(String name, Object value): Set an attribute in the context.//test level variable
	• getSuite().setAttribute(String name, Object value): Set an attribute in the suite.//suite level variable
	• getAttribute(String name): Retrieve an attribute from the context
	
	``
	@BeforeTest
	public void SetData(ITestContext context){
	
	String Customer_id = "C11012034";
	context.setAttribute("CustID", Customer_id);
	System.out.println("Value is stored in ITestContext");
	System.out.println("++");
	
	}
	@Test
	public void Test1a(ITestContext context){
	String Customer_id1 = (String) context.getAttribute("CustID");
	System.out.println("In Test1, Value stored in context is: "+Customer_id1);
	System.out.println("++");
	
	}
	``

ITestListener	ITestListener is an interface implemented in the class, and that class overrides the ITestListener-defined methods. It listens to the desired events and executes the methods accordingly. • onStart(): invoked after test class is instantiated and before execution of any testNG method. • onTestSuccess(): invoked on the success of a test • onTestFailure(): invoked on the failure of a test • onTestSkipped(): invoked when a test is skipped • onTestFailedButWithinSuccessPercentage(): invoked whenever a method fails but within the defined success percentage • onFinish(): invoked after all tests of a class are executedThe above-mentioned methods use the parameters ITestContext and ITestResult. The ITestContext is a class that contains information about the test run. The ITestResult is an interface that defines the result of the test.

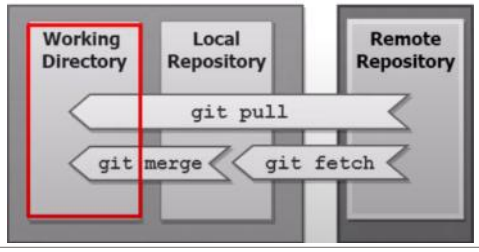
	import org.testng.ITestContext;
	import org.testng.ITestListener;
	import org.testng.ITestResult;
	
	public class MyListener implements ITestListener {
	
	@Override
	public void onFinish(ITestContext contextFinish) {
	System.out.println("onFinish method finished");
	
	}
	
	@Override
	public void onStart(ITestContext contextStart) {
	System.out.println("onStart method started");
	}
	
	@Override
	public void onTestFailedButWithinSuccessPercentage(ITestResult result) {
	System.out.println("Method failed with certain success percentage"+ result.getName());
	
	}
	
	@Override
	public void onTestFailure(ITestResult result) {
	System.out.println("Method failed"+ result.getName());
	
	}
	
	@Override
	public void onTestSkipped(ITestResult result) {
	System.out.println("Method skipped"+ result.getName());
	
	}
	
	@Override
	public void onTestStart(ITestResult result) {
	System.out.println("Method started"+ result.getName());
	
	}
	
	@Override
	public void onTestSuccess(ITestResult result) {
	System.out.println("Method passed"+ result.getName());
	
	}
	}

AutoRead Prop Files	Default location src/test/resources ResourceBundle.getBundle("routes");

Pass Data from xml to cucumber runner	public class TestRunner extends AbstractTestNGCucumberTests {
	
	  @Override
	  @DataProvider(parallel = false)
	  public Object[][] scenarios() {
	  return super.scenarios();
	  }
	
	  @Parameters({ "browser" })
	  @BeforeMethod
	  public static void setUpScenario(String browser) {
	  WebDriverManager.initDriver(browser);
	  }
	
	  @AfterMethod
	  public void tearDown(){
	  WebDriverManager.quitBrowser();
	  }
	}

Git

29 November 2024 08:21

Command	Description
git init	Initializes a new Git repository in the current directory.
git clone <url>	Clones an existing Git repository from 1 a remote URL.
git add <file>	Adds a file to the staging area.
git commit -m "<message>"	Commits the staged changes with a message.
git status	Shows the current state of the working directory.
git diff	Shows the difference between the current state and the previous commit.
git log	Shows the commit history.
git checkout <branch>	Switches to a different branch.
git branch <branch-name>	Creates a new branch.
git checkout -b <name>	creates and switches
git checkout commitHash	used to switch your working directory to the state of a specific commit. This allows you to view or inspect a project's state at a particular point in history. However, this action places you in a detached HEAD state, meaning you're not on any branch. You are not on any branch. Any changes made here will not be associated with a branch unless you explicitly create one. If you make changes and want to keep them, create a new branch: git checkout -b new-branch-name Modern Alternative: git switch --detach <commitHash>
git merge <branch>	Merges a branch into the current branch. git checkout master git pull origin master git checkout subBranch git merge master
ggit rebase -i HEAD~2	Combines multiple commits into a single commit. add squash text before other commits to be merged
git push <remote> <branch> -f to force when you revert commit in local	Pushes changes to a remote repository.
git pull <remote> <branch>	Pulls changes from a remote repository.
git remote add <name> <url>	Adds a remote repository.
git remote show <name>	Shows information about a remote repository.
git fetch <remote>	Fetches changes from a remote repository to local repository. Need to merge in order to see changes in working directory. 
git stash	Temporarily saves changes to the working directory.
git stash pop	Restores the most recently stashed changes.
git config --global user.name "<name>"	Sets the global user name.
git config --global user.email "<email>"	Sets the global user email.
git config --global core.editor "<editor>"	Sets the global default editor.
Delete untracked files	git clean -f
git reset	Staging to Working Directory

git reset --soft HEAD~1	revert commit and changes move to staging, hard will delete permanently. after ~ we can also give commit hash above which we want to delete things we can also give commit hash
git reset --hard HEAD~1	Revert commit and delete data
git revert <commit-hash> git revert <commit-hash1> <commit-hash2>	git revert is used to undo a specific commit by creating a new commit that reverses the changes made by the original commit. Unlike git reset, which can rewrite history, git revert is a safe operation because it doesn't alter the commit history, making it ideal for collaborative workflows. For Conflicts git add <file> git revert --continue git push
git bisect start git bisect bad [<commit>] git bisect good [<commit>]	used to perform a binary search through a repository's commit history to identify the commit that introduced a bug or issue. It is particularly useful in large repositories with numerous commits, as it narrows down the problematic commit efficiently. The process involves marking a good commit (where the code works) and a bad commit (where the bug exists). Git then automatically checks out commits in the middle of the range for you to test, helping narrow down the problematic commit. Git will now check out a commit halfway between the good and bad commits. Test this commit to determine if it’s good or bad: If the commit is bad, mark it as:git bisect bad else git bisect good
git commit --amend -m "New Message"	Fix a Commit Message
git add <file> git commit --amend	Add Changes to the Last Commit
git reset --soft HEAD~1 git restore --staged <file> git commit --amend	Remove Changes from the Last Commit

Aspect	Merge	Rebase	Squash
History	Preserves branch history with merge commits	Creates a linear history. Reapplies commits from one branch on top of another, creating a linear history by rewriting commit history.	Combines multiple commits into one
Commit Granularity	Keeps all original commits intact Creates One more commit	Keeps individual commits intact No Additional Commits	Reduces all commits to a single one
Complexity	Simple, no rewriting	More complex, rewrites history	Requires manual consolidation
Risk of Conflicts	During the merge process only	During the rebase process	During squashing, if conflicts exist
Use in Shared Branches	Safe	Risky	Risky unless handled carefully
Clean History	Can be messy	Clean and linear	Very clean and concise

Maven

16 February 2024 08:36

1. Settings.xml is used to modify behavior of Maven
2. Bin Folder: Contains scripts for executing Maven.
3. Boot: Contains a jar file responsible for creating a class loader for Maven execution
4. Conf: This has settings.xml which when customised is paced under m2 directory
5. Lib: Contains Maven core classes for building.Includes Maven dependencies and Java components necessary for Maven's operation.

   mvn archetype:generate -DarchetypeGroupId=org.apache.maven.archetypes -DarchetypeArtifactId=maven-archetype-quickstart -DgroupId=com.example -DartifactId=my-project -Dversion=1.0-SNAPSHOT -DinteractiveMode=false

   archetype:generate is Maven Goal where archetype is plugin and generate is GoalID

   Maven Coordinates: Group id, artifact id, packaging and Version

   Maven goals represent individual tasks that can be executed during the build process, while plugins are extensions that provide additional functionality and encapsulate collections of goals. Together, goals and plugins enable Maven to efficiently manage and automate the build lifecycle of a project

   Scopes
   1. Compile is default scope:Available in all classpaths and propagated to dependent projects
   2. test: Only available for test compilation and execution phases
   3. provided: Available during compilation and testing, but not at runtime. Not for packaging
   4. runtime: Not available during compilation, but available at runtime. Not for Compilation